

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent
Kind Code
Date of Patent
Inventor(s)

12407693
B2
September 02, 2025
Covaci; Alexandra et al.

System for securing verification key from alteration and verifying validity of a proof of correctness

Abstract

The invention relates to distributed ledger technologies such as consensus-based blockchains. A blockchain transaction may include digital resources that are encumbered by a locking script that encodes a set of conditions that must be fulfilled before the encumbered resources may be used (e.g., transferring ownership/control of encumbered resources). A worker (e.g., a computer system) performs one or more computations to generate a proof, which is encoded as part of an unlocking script. A verification algorithm may utilize the proof, a verification key, and additional data such as a cryptographic material associated with the worker (e.g., a digital signature) to verify that digital assets of the transaction should be transferred. As a result of the validation of this transaction, any third party is able to check the contract was executed corrected rather than re-executing the contract, thus saving computational power.

Inventors: Covaci; Alexandra (Canterbury, GB), Madeo; Simone (London, GB), Motylinski; Patrick (London, GB), Vincent; Stephane (Luxembourg, LU)

Applicant: nChain Licensing AG (Zug, CH)

Family ID: 64100702

Assignee: NCHAIN LICENSING AG (Zug, CH)

Appl. No.: 18/667910

Filed: May 17, 2024

Prior Publication Data

Document Identifier	Publication Date
US 20240313947 A1	Sep. 19, 2024

Foreign Application Priority Data

GB	1718505	Nov. 09, 2017
GB	1719998	Nov. 30, 2017
GB	1720768	Dec. 13, 2017
GB	1801753	Feb. 02, 2018
GB	1805948	Apr. 10, 2018
GB	1806444	Apr. 20, 2018

Related U.S. Application Data

continuation parent-doc US 18134276 20230413 US 12219044 child-doc US 18667910
continuation parent-doc US 16762471 US 11658801 20230523 WO PCT/IB2018/058433 20181029 child-doc US 18134276

Publication Classification

Int. Cl.: H04L29/06 (20060101); G06F8/41 (20180101); G06F9/445 (20180101); G06F16/27 (20190101); G06Q20/06 (20120101); G06Q20/40 (20120101); H04L9/00 (20220101); H04L9/06 (20060101); H04L9/30 (20060101); H04L9/32 (20060101); H04L9/40 (20220101)

U.S. Cl.:

CPC H04L63/12 (20130101); G06F8/44 (20130101); G06F8/447 (20130101); G06F8/451 (20130101); G06F9/44521 (20130101); G06F16/27 (20190101); G06Q20/065 (20130101); G06Q20/401 (20130101); G06Q20/405 (20130101); H04L9/0637 (20130101); H04L9/0643 (20130101); H04L9/30 (20130101); H04L9/3218 (20130101); H04L9/3236 (20130101); H04L9/3239 (20130101); H04L9/3247 (20130101); H04L9/3263 (20130101); H04L9/50 (20220501); H04L9/0618 (20130101); H04L9/3066 (20130101); H04L2209/34 (20130101); H04L2209/56 (20130101)

Field of Classification Search

USPC: None

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
4667290	12/1986	Goss et al.	N/A	N/A
5297150	12/1993	Clark	N/A	N/A
5404531	12/1994	Wakatani	N/A	N/A
5499191	12/1995	Young	N/A	N/A
5920830	12/1998	Hatfield et al.	N/A	N/A
6064928	12/1999	Wilson et al.	N/A	N/A
6161180	12/1999	Matyas et al.	N/A	N/A
6519754	12/2002	McElvain et al.	N/A	N/A
7085701	12/2005	Rich et al.	N/A	N/A
7209555	12/2006	Futa et al.	N/A	N/A
7281017	12/2006	Hostetter et al.	N/A	N/A
7590236	12/2008	Boneh et al.	N/A	N/A
8165287	12/2011	Ghouti et al.	N/A	N/A
8189771	12/2011	Ghouti et al.	N/A	N/A
8331556	12/2011	Billet et al.	N/A	N/A
8607129	12/2012	Radhakrishnan et al.	N/A	N/A
8824670	12/2013	Icart et al.	N/A	N/A
8904181	12/2013	Felsher et al.	N/A	N/A
9026978	12/2014	Liu et al.	N/A	N/A
9286602	12/2015	Rosati et al.	N/A	N/A
9483596	12/2015	Badar et al.	N/A	N/A
9569771	12/2016	Lesavich et al.	N/A	N/A
10135607	12/2017	Roets	N/A	N/A
10339523	12/2018	McDonough et al.	N/A	N/A
10419209	12/2018	Griffin et al.	N/A	N/A
10515567	12/2018	Lablans	N/A	N/A
11226799	12/2021	Sundaresan et al.	N/A	N/A
2003/0125917	12/2002	Rich et al.	N/A	N/A
2004/0015739	12/2003	Heinkel et al.	N/A	N/A
2005/0004899	12/2004	Baldwin et al.	N/A	N/A
2005/0139657	12/2004	Hopkins	N/A	N/A
2005/0262353	12/2004	Gentry et al.	N/A	N/A
2006/0149962	12/2005	Fountain et al.	N/A	N/A
2007/0061487	12/2006	Moore et al.	N/A	N/A
2007/0157132	12/2006	Cheng et al.	N/A	N/A
2008/0127067	12/2007	Aubertine et al.	N/A	N/A
2010/0067686	12/2009	Minematsu	N/A	N/A
2010/0131933	12/2009	Kim et al.	N/A	N/A
2010/0272209	12/2009	Lee et al.	N/A	N/A
2011/0024678	12/2010	Schmitt	N/A	N/A
2011/0200188	12/2010	Ghouti et al.	N/A	N/A
2012/0284175	12/2011	Wilson et al.	N/A	N/A
2013/0031446	12/2012	Kamiya	N/A	N/A
2013/0097420	12/2012	Zaverucha	N/A	N/A
2014/0250296	12/2013	Hansen	N/A	N/A
2014/0321644	12/2013	Lemieux	N/A	N/A
2014/0337234	12/2013	Tang et al.	N/A	N/A
2015/0363598	12/2014	Xu et al.	N/A	N/A
2015/0379510	12/2014	Smith	N/A	N/A
2016/0004820	12/2015	Moore	N/A	N/A
2016/0087802	12/2015	Peeters	N/A	N/A
2016/0140340	12/2015	Walters et al.	N/A	N/A
2016/0162897	12/2015	Feeney	N/A	N/A
2016/0204938	12/2015	Kounavis et al.	N/A	N/A
2016/0283941	12/2015	Andrade	N/A	N/A
2016/0357948	12/2015	Takeuchi	N/A	N/A
2017/0039330	12/2016	Tanner, Jr. et al.	N/A	N/A
2017/0048209	12/2016	Lohe et al.	N/A	N/A
2017/0085545	12/2016	Lohe et al.	N/A	N/A
2017/0091750	12/2016	Maim	N/A	N/A
2017/0091756	12/2016	Stern et al.	N/A	N/A
2017/0131983	12/2016	Roytman et al.	N/A	N/A
2017/0132421	12/2016	Unitt	N/A	N/A
2017/0132619	12/2016	Miller et al.	N/A	N/A
2017/0140408	12/2016	Wuehler	N/A	N/A
2017/0142103	12/2016	Bringer et al.	N/A	N/A
2017/0155515	12/2016	Androulaki et al.	N/A	N/A
2017/0177312	12/2016	Boehm et al.	N/A	N/A
2017/0178263	12/2016	Kraemer et al.	N/A	N/A
2017/0180341	12/2016	Walker et al.	N/A	N/A

2017/0220815	12/2016	Ansari et al.	N/A	N/A
2017/0221052	12/2016	Sheng et al.	N/A	N/A
2017/0249716	12/2016	Meixner et al.	N/A	N/A
2017/0250815	12/2016	Cuende et al.	N/A	N/A
2017/0277909	12/2016	Kraemer et al.	N/A	N/A
2017/0278100	12/2016	Kraemer et al.	N/A	N/A
2017/0279611	12/2016	Kraemer et al.	N/A	N/A
2017/0286079	12/2016	Cho et al.	N/A	N/A
2017/0286717	12/2016	Khi et al.	N/A	N/A
2017/0287090	12/2016	Hunn et al.	N/A	N/A
2017/0317833	12/2016	Smith et al.	N/A	N/A
2017/0317834	12/2016	Smith et al.	N/A	N/A
2017/0337319	12/2016	Camus et al.	N/A	N/A
2017/0338947	12/2016	Ateniese et al.	N/A	N/A
2017/0344988	12/2016	Cusden et al.	N/A	N/A
2017/0352209	12/2016	Keuffer et al.	N/A	N/A
2017/0353309	12/2016	Gray	N/A	N/A
2018/0034634	12/2017	Benarroch Guenun et al.	N/A	N/A
2018/0039667	12/2017	Pierce et al.	N/A	N/A
2018/0049043	12/2017	Hoffberg	N/A	N/A
2018/0075453	12/2017	Durvasula et al.	N/A	N/A
2018/0089758	12/2017	Stradling et al.	N/A	N/A
2018/0117446	12/2017	Tran et al.	N/A	N/A
2018/0167201	12/2017	Naqvi	N/A	N/A
2018/0204005	12/2017	Gajek et al.	N/A	N/A
2018/0270065	12/2017	Brown et al.	N/A	N/A
2018/0349631	12/2017	Illendula et al.	N/A	N/A
2019/0095631	12/2018	Roets et al.	N/A	N/A
2019/0138753	12/2018	Wallrabenstein	N/A	N/A
2019/0163887	12/2018	Frederick et al.	N/A	N/A
2019/0180276	12/2018	Lee et al.	N/A	N/A
2019/0182035	12/2018	Chari et al.	N/A	N/A
2019/0295182	12/2018	Kfir et al.	N/A	N/A
2020/0050780	12/2019	Uhr et al.	N/A	N/A
2020/0184557	12/2019	Wang	N/A	N/A
2020/0327498	12/2019	Weber et al.	N/A	N/A
2021/0019746	12/2020	Adjaz et al.	N/A	N/A
2021/0073795	12/2020	Ruiz et al.	N/A	N/A
2021/0342490	12/2020	Briancon et al.	N/A	N/A
2023/0269070	12/2022	Covaci et al.	N/A	N/A
2024/0143739	12/2023	Cao et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
104580240	12/2014	CN	N/A
106506146	12/2016	CN	N/A
106534317	12/2016	CN	N/A
107040545	12/2016	CN	N/A
107179932	12/2016	CN	N/A
107274184	12/2016	CN	N/A
107426234	12/2016	CN	N/A
3249599	12/2016	EP	N/A
H08305547	12/1995	JP	N/A
2006505055	12/2005	JP	N/A
2009541853	12/2008	JP	N/A
2011119952	12/2010	JP	N/A
5697153	12/2014	JP	N/A
2015132754	12/2014	JP	N/A
7208989	12/2022	JP	N/A
7221954	12/2022	JP	N/A
7285840	12/2022	JP	N/A
101795696	12/2016	KR	N/A
20200096790	12/2019	KR	N/A
2016131577	12/2015	WO	N/A
2016155804	12/2015	WO	N/A
2016206567	12/2015	WO	N/A
2017008829	12/2016	WO	N/A
2017032541	12/2016	WO	N/A
2017079652	12/2016	WO	N/A
2017104149	12/2016	WO	N/A
2017145010	12/2016	WO	N/A
2017148527	12/2016	WO	N/A
2017178956	12/2016	WO	N/A
2017187396	12/2016	WO	N/A
2017187398	12/2016	WO	N/A

2017187399	12/2016	WO	N/A
2017190795	12/2016	WO	N/A
2018127446	12/2017	WO	N/A
2018127456	12/2017	WO	N/A

OTHER PUBLICATIONS

Jakob Eberhardt: “ZoKrates—A Toolbox for zkSNARKs on Ethereum”, Youtube, Nov. 4, 2017 (Nov. 4, 2017), p. 1 pp., XP054979013, Retrieved from the Internet: URL:https://www.youtube.com/watch?v=sSlryw_b5J0&t=4s (Year: 2017). cited by examiner

Andreas M. Antonopoulos: “Mastering Bitcoin—Unlocking Digital Cryptocurrencies” In: “Mastering bitcoin : [unlocking digital cryptocurrencies]”, Dec. 20, 2014 (Dec. 20, 2014), O'Reilly Media, Beijing Cambridge Farnham Koln Sebastopol Tokyo, XP055306939, ISBN: 978-1-4493-7404-4 (Year: 2014). cited by examiner

Matteo Campanelli et al.: “Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services”, IACR, International Association for Cryptologic Research, vol. 20171101:190916 Jun. 9, 2017 (Jun. 9, 2017), pp. 1-26, XP061023635, Retrieved from the Internet: (Year: 2017). cited by examiner

Menezes et al., “Handbook of Applied Cryptography: pp. 33, 38,” CRC Press, Oct. 16, 1996, 3 pages. cited by applicant

Sanchez, David Cerezo, “Raziel: Private and Verifiable Smart Contracts in Blockchains”, Sep. 17, 2017, 55 pages. cited by applicant

Seijas et al., “Scripting Smart Contract for Distributed Ledger Technology”, Feb. 10, 2017, 33 pages. cited by applicant

UK IPO Search Report mailed Nov. 8, 2018, Patent Application No. GB1806444.4, 6 pages. cited by applicant

United Kingdom Commercial Search Report mailed Apr. 20, 2018, Patent Application No. 1720768.9, filed Dec. 13, 2017, 8 pages. cited by applicant

United Kingdom Intellectual Property Office Search Report mailed Jun. 12, 2018, Patent Application No. 1720768.9, filed Dec. 13, 2017, 7 pages. cited by applicant

United Kingdom Intellectual Property Office Search Report mailed May 3, 2018, Patent Application No. 1719998.5, filed Nov. 30, 2017, 6 pages. cited by applicant

United Kingdom IPO Search Report mailed Apr. 27, 2018, Patent Application No. 1718505.9, filed Nov. 9, 2017, 5 pages. cited by applicant

Van Den Hooff et al., “VerSum: Verifiable Computations over Large Public Logs,” Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Nov. 3, 2014, 14 pages. cited by applicant

Vangie, Beal, “What is a Computer System?” Webopedia, Aug. 1, 2022, <https://www.webopedia.com/definitions/computer-system>, 2 pages. cited by applicant

Viacoin Dev Team, “Styx: Unlinkable Anonymous Atomic Payment Hub for Viacoin,” viacoin.org, Oct. 14, 2016, <http://docplayer.net/35213119-Styx-unlinkable-anonymous-atomic-payment-hub-for-viacoin-viacoin-dev-team-viacoin-org.html>, 18 pages. cited by applicant

Virza, “On Deploying Succinct Zero-Knowledge Proofs” [online] MIT, Sep. 2017 [retrieved Feb. 10, 2022]. Retrieved from the Internet: URL: On Deploying Succinct Zero-Knowledge Proofs, 2016, 131 pages. cited by applicant

Wikipedia, “Huffman coding,” Wikipedia the Free Encyclopedia, Feb. 24, 2018, https://en.wikipedia.org/w/index.php?title=Huffman_coding&oldid=827366029, 11 pages. cited by applicant

Wikipedia, “Precompiled Header,” Retrieved Mar. 30, 2022, https://en.wikipedia.org/w/index.php?title=Precompiled_header&oldid=807155683, 3 pages. cited by applicant

Wikipedia, “Zero Knowledge Contingent Payment,” Bitcoin Wiki, retrieved from https://en.bitcoin.it/wiki/Zero_Knowledge_Contingent_Payment, Apr. 8, 2020, 3 pages. cited by applicant

Wu et al., “Libsnark: a C++ Library for zkSNARK Proofs,” SCIPR Lab, libsnark/README.md at 92a80f74727091fdc40e6021dc42e9f6b67d5176, Aug. 18, 2017, 9 pages. cited by applicant

Zcash, “zk-SNARKs,” zCash website, retrieved Apr. 1, 2022 from <https://web.archive.org/web/20171107012237/https://z.cash/technology/zksnarks.html>, Nov. 24, 2017, 1 page. cited by applicant

Zhang et al., “A Remote-Attestation-Based Extended Hash Algorithm for Privacy Protection,” 2017 International Conference on Computer Network, Electronic and Automation (ICCNEA), Sep. 23, 2017, 4 pages. cited by applicant

Zyskind et al., “Enigma: Decentralized Computation Platform with Guaranteed Privacy,” arXiv preprint arXiv:1506, Jun. 10, 2015, 14 pages. cited by applicant

Intellectual Property Office of Korea, “Request for the Submission of an Opinion” in Application No. 10-2024-7024191, Aug. 5, 2024, 4 pages. cited by applicant

Shocker, Ali, “Sustainable Blockchain through Proof of exercise”, 2017 IEEE 16th International Symposium on Network Computing and Applications (NCA), Nov. 1, 2017, 9 pages. cited by applicant

Ball, M. et al., “Proof of Useful Work”, Cryptology ePrint Archive, Report, Feb. 27, 2017, 31 pages. cited by applicant

“How Log Proofs Work,” Certificate Transparency, Sep. 25, 2017 [retrieved May 2, 2018], <https://web.archive.org/web/20170925180136/http://www.certificate-transparency.org/log-proofs-work>, 5 pages. cited by applicant

Anonymous, “Background Page,” Oraclize, Oct. 2017 [retrieved May 2, 2018], <https://web.archive.org/web/20171017121053/http://docs.oracize.it/>, 18 pages. cited by applicant

Antonopoulos, “Mastering Bitcoin—Unlocking Digital Cryptocurrencies,” O'Reilly Media, Inc., Dec. 20, 2014, 282 pages. cited by applicant

Ben-Sasson et al. “Scalable Zero Knowledge via Cycles of Elliptic Curves” [online] IACR, Sep. 18, 2016 [retrieved Feb. 10, 2022]. Retrieved from <https://eprint.iacr.org/2014/595.pdf>, 2016, 49 pages. cited by applicant

Ben-Sasson et al., “SNARKs for C: Verifying program executions succinctly and in zero knowledge,” Advances in Cryptology—Crypto 2013, Aug. 18, 2013, 19 pages. cited by applicant

Ben-Sasson et al., “Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture,” USENIX Security 2014, first disclosed Dec. 30, 2013, last revised May 19, 2015, <https://eprint.iacr.org/2013/879.pdf>, 37 pages. cited by applicant

Ben-Sasson et al., “Zerocash: Decentralized Anonymous Payments from Bitcoin,” 2014 IEEE Symposium on Security and Privacy, May 18, 2014, <http://zerocash-project.org/media/pdf/zerocash-oakland2014.pdf>, 16 pages. cited by applicant

Bertani et al., “How can I trust smart contracts that use Oraclize?,” Reddit, May 25, 2017 [retrieved May 2, 2018], https://www.reddit.com/r/ethereum/comments/6d7j7x/how_can_i_trust_smart_contracts_that_use_oracize/di0nb17/, 5 pages. cited by applicant

Bertani, “Scalable Onchain Verification for Authenticated Data Feeds and Offchain Computations,” YouTube, Ethereum Foundation, Nov. 26, 2017 [retrieved May 10, 2018], <https://www.youtube.com/watch?v=7uQdEBVu8Sk>, 19:19, 4 pages. cited by applicant

Bitcoin Core, “The First Successful Zero-Knowledge Contingent Payment”, 2022, 5 pages. cited by applicant

Bitcoinstrings, “Blockchain in Words,” retrieved from <https://bitcoinstrings.com/blk00281.txt>, Dec. 2013, 667 pages. cited by applicant

Bowe, “Pay-to-Sudoku,” GitHub, retrieved from <https://github.com/zcash-hackworks/pay-to-sudoku/blob/master/README.md>, 2016, 2 pages. cited by applicant

Brown et al., “Transport layer security (tls) evidence extensions,” Working Draft, IETF Secretariat, Internet-Draft draft-housley-evidence-extns-01, <https://tools.ietf.org/pdf/draft-housley-evidence-extns-01>, Nov. 2006 [retrieved May 2, 2018], 21 pages. cited by applicant

Buterin, "Quadratic Arithmetic Programs: from Zero to Hero," retrieved from <https://medium.com/@Vitalik/quadratic-arithmetic-programs-from-zero-to-hero-f6d558cea649>, Dec. 11, 2016, 9 pages. cited by applicant

Campanelli et al., "Zero-knowledge contingent payments revisited: Attacks and payments for services," Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, Oct. 30, 2017, 28 pages. cited by applicant

Canetti et al., "Practical Delegation of Computation Using Multiple Servers," CCS, Oct. 17, 2011, 10 pages. cited by applicant

Castor, "Trust Your Oracle? Cornell Launches Tool for Confidential Blockchain Queries," CoinDesk, retrieved from <https://www.coindesk.com/tech/2017/05/17/trust-your-oracle-cornell-launches-tool-for-confidential-blockchain-queries/>, May 17, 2017, 5 pages. cited by applicant

Chen et al., "Algebraic Geometric Secret Sharing Schemes and Secure Multi-Party Computations over Small Fields", Advances in Cryptology, 2006, 16 pages. cited by applicant

Commercial Search Report mailed Feb. 28, 2018, United Kingdom Patent Application No. 1719998.5, filed Nov. 30, 2017, 6 pages. cited by applicant

Costello et al., "Geppetto: versatile Verifiable Computation," 2015 IEEE Symposium on Security and Privacy, 2015, 22 pages. cited by applicant

Covaci et al., "Computer-implemented system and method," United Kingdom Patent Application No. 1720946.1, filed Dec. 15, 2017. cited by applicant

Covaci et al., "Extracting Information from the CRS in a ZK Protocol on Blockchain," United Kingdom Patent Application No. 1719998.5, filed Nov. 30, 2017, 39 pages. cited by applicant

Covaci et al., "Logic Minimisation of C-like Smart Contracts for Optimised Verifiable Computation," United Kingdom Patent Application No. 1718505.9, filed Nov. 9, 2017, 38 pages. cited by applicant

Covaci et al., "Nectar: Non-Interactive Smart Contract Protocol using Blockchain Technology," arXiv preprint arXiv:1803.04860, Mar. 13, 2018, 8 pages. cited by applicant

Davidson et al., "Empowering the Economy of Things," 2017, 54 pages. cited by applicant

Eberhardt et al., "ZoKrates—Scalable Privacy-Preserving Off-Chain Computations," Information Systems Engineering (ISE), Aug. 2018, 8 pages. cited by applicant

Ethereum Foundation, "ZoKrates—A Toolbox for zkSNARKS on Ethereum," https://www.youtube.com/watch?v=sSlryw5J_0, Nov. 26, 2017, 12 pages. cited by applicant

Fee et al., "Cryptography using Chebyshev polynomials," Maple Summer Workshop, Burnaby, Canada, Jul. 11, 2004, <http://www.cecm.sfu.ca/CAG/ppaers/CHEB>, 16 pages. cited by applicant

Fiore et al., Hash First, Argue Later Adaptive Verifiable Computations on Outsourced Data, ACM Computer and Communications Security, 2016, 40 pages. cited by applicant

Fournet et al., "A Certified Compiler for Verifiable Computing," HAL Open Science, Jun. 2016, 14 pages. cited by applicant

Franz et al., "CBMC-GC: An ANSI C Compiler for Secure Two-Party Computations," retrieved from https://arise.or.at/pubpdf/CBMC-GC_An_ANSI_C_Compiler_for_Secure_Two-Party_Computations.pdf, 2014, 5 pages. cited by applicant

Fu, "Off-Chain Computation Solutions for Ethereum Developers," Medium, Sep. 12, 2017 [retrieved May 2, 2018], <https://medium.com/@YondonFu/off-chain-computation-solutions-for-ethereum-developers-507b23355b17>, 8 pages. cited by applicant

Fuchsbaauer et al., "Proofs on Encrypted Values in Bilinear Groups and an Application to Anonymity of Signatures," Third International Conference on Pairing-based Cryptography, Aug. 2009, 26 pages. cited by applicant

Gennaro et al., "Quadratic Span Programs and Succinct NIZKs without PCPs," Annual International Conference on the Theory and Applications of Cryptographic Techniques, May 26, 2013, 20 pages. cited by applicant

Gennaro et al., "Robust Threshold DSS Signatures," International Conference on the Theory and Applications of Cryptographic Techniques, May 12, 1996, https://link.springer.com/content/pdf/10.1007%2F3-540-68339-9_31.pdf, 18 pages. cited by applicant

Goldfeder et al., "Escrow Protocols for Cryptocurrencies: How to Buy Physical Goods Using Bitcoin," retrieved from <http://stevengoldfeder.com/papers/escrow.pdf>, Jul. 26, 2018, 27 pages. cited by applicant

Hajjeh et al., "TLS Sign," TLS Working Group, Internet Draft Version 4, Dec. 15, 2007 [retrieved May 2, 2018], <https://tools.ietf.org/html/draft-hajjeh-tls-sign-04>, 12 pages. cited by applicant

Hearn, "Continuing the zkSNARK Tutorials," retrieved from <https://blog.plan99.net/vntinyram-7b9d5b299097>, Dec. 15, 2016, 9 pages. cited by applicant

Hong et al., "Verifiable Computation of Large Polynomials," retrieved from <http://or.nsf.gov.cn/bitstream/00001903-5/154735/1/1000009080185.pdf>, Dec. 16, 2014, 13 pages. cited by applicant

Intellectual Property Office of Korea, "Request for the Submission of an Opinion" in Application No. 10-2020-7014213, Jan. 31, 2024, 7 pages. cited by applicant

International Search Report and Written Opinion mailed Jan. 15, 2019, Patent Application No. PCT/IB2018/058434, 11 pages. cited by applicant

International Search Report and Written Opinion mailed Jan. 17, 2019, Patent Application No. PCT/IB2018/058432, 11 pages. cited by applicant

International Search Report and Written Opinion mailed Jan. 17, 2019, Patent Application No. PCT/IB2018/058437, 10 pages. cited by applicant

International Search Report and Written Opinion mailed Jan. 22, 2019, Patent Application No. PCT/IB2018/058583, 10 pages. cited by applicant

International Search Report and Written Opinion mailed Jan. 23, 2019, Patent Application No. PCT/IB2018/058433, 12 pages. cited by applicant

International Search Report and Written Opinion mailed Jan. 23, 2019, Patent Application No. PCT/IB2018/058491, 12 pages. cited by applicant

International Search Report and Written Opinion mailed Mar. 14, 2019, Patent Application No. PCT/IB2018/059770, 12 pages. cited by applicant

International Search Report and Written Opinion mailed Mar. 14, 2019, Patent Application No. PCT/IB2018/059920, 12 pages. cited by applicant

International Search Report and Written Opinion mailed Mar. 19, 2019, Patent Application No. PCT/IB2018/059918, 14 pages. cited by applicant

Japan Patent Office "Notice of Reasons for Rejection" in Application No. 2023-014319, Feb. 13, 2024, 12 pages. cited by applicant

Jehan, "Rockchain Decentralized Audited Data Networks," White Paper, retrieved from <https://www.rockchain.org/RockchainWhitePaper.pdf>, Jan. 20, 2018, 28 pages. cited by applicant

Justus, Benjamin, "Point Compression and Coordinate Recovery for Edwards Curves over Finite Field", Annals of West University of Timisoara-Mathematics and Computer Science, 2014, 15 pages. cited by applicant

Kerber, "Verifiable Computation in Smart Contracts," University of Edinburgh School of Informatics Computer Science 4th Year Project Report, published online Apr. 4, 2017 [retrieved May 2, 2018], <https://git.drwx.org/bsc/proj-report/raw/branch/master/report.pdf>, 49 pages. cited by applicant

Keutzer et al., "Anatomy of a Hardware Compiler," 1988, 10 pages. cited by applicant

Kiaiyas et al., "Proofs of Proofs of Work with Sublinear Complexity," Financial Cryptography and Data Security, 2016, 18 pages. cited by applicant

Klmoney, "Part 1: Transaction Basics", Jun. 6, 2017, retrieved from the internet, <https://web.archive.org/web/20170606202729/https://klmoney.wordpress.com/bitcoin-dissecting-transactions-part-1>, 9 pages. cited by applicant

Kosba et al., "Hawk: The Blockchain Model of Cryptography and Privacy-Preserving Smart Contracts," IEEE Symposium on Security and Privacy, May 22, 2016, 31 pages. cited by applicant

Kreuter, "Techniques for Scalable Secure Computation Systems," retrieved from <https://repository.library.northeastern.edu/files/neu:cj82rh04k/fulltext.pdf>, May 2018, 145 pages. cited by applicant

Kröl et al., "SPOC: Secure Payments to Outsourced Computations," Jul. 17, 2018, 6 pages. cited by applicant

Kumaresan et al., "How to Use Bitcoin to Incentivize Correct Computations," retrieved from <https://people.csail.mit.edu/ranjit/papers/incentives.pdf>, Nov. 2014, 12 pages. cited by applicant

Kuzminov et al., "Bridging the Gaps with Iolite Blockchain," Iolite Foundation White Paper, 2017, 13 pages. cited by applicant

Malkhi et al., "Fairplay—A Secure Two-Party Computation System," Aug. 2004, 17 pages. cited by applicant

Mathworks, "Matlab Coder—Generate C and C++ Code From Matlab Code," 9 pages. cited by applicant

Maxwell et al., "Chat logs," Bitcoin-wizards IRC Chat Channel, Aug. 16, 2013 [retrieved May 2, 2018], <https://download.wpsoftware.net/bitcoin/wizards/2013/08/13-08-16.log>, 1 page. cited by applicant

Maxwell et al., "CoinCovenants using SCIP signatures, an amusingly bad idea," Bitcoin Forum, Aug. 20, 2013 [retrieved Apr. 13, 2018], <https://bitcointalk.org/index.php?topic=278122.0>, 5 pages. cited by applicant

Maxwell et al., "Really Really ultimate blockchain compression: CoinWitness," Bitcoin Forum, Aug. 19, 2013 [retrieved Apr. 11, 2018], <https://bitcointalk.org/index.php?topic=277389.0>, 7 pages. cited by applicant

Maxwell, "The First Successful Zero-Knowledge Contingent Payment," Bitcoin Core, retrieved from <https://bitcoincore.org/en/2016/02/26/zero-knowledge-contingent-payments-announcement/>, Feb. 26, 2016, 5 pages. cited by applicant

Mayer, "zk-Snark Explained: Basic Principles," Dec. 13, 2016, 9 pages. cited by applicant

Mayer, "zk-Snark explained: Basic Principles," retrieved from https://www.researchgate.net/publication/321124635_zk-SNARK_explained_Basic_Principles, Dec. 2016, 9 pages. cited by applicant

Müller, "A Short Note on Secret Sharing Using Elliptic Curves," Proceedings of Secrypt 2008, Jul. 26, 2008, <http://www.scitepress.org/Papers/2008/19183/19183.pdf>, 4 pages. cited by applicant

Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Bitcoin, Oct. 31, 2008, <https://bitcoin.org/bitcoin.pdf>, 9 pages. cited by applicant

Ning, "Automatically Convert Matlab Code to C Code," <https://www.mathworks.com/videos/automatically-converting-matlab-code-to-c-code-96483.html>, Aug. 19, 2014, 8 pages. cited by applicant

Paganini, Pierluigi What is a Digital Signature? Fundamental Principles, Security Affairs, May 2012, <https://securityaffairs.com/5223/digital-id/what-is-a-digital-signature-fundamental-principles.html>, 7 pages. cited by applicant

Parno et al., "Pinocchio: Nearly Practical Verifiable Computation," IEEE Symposium on Security and Privacy, May 19, 2013, 16 pages. cited by applicant

Parno et al., "Pinocchio: Nearly Practical Verifiable Computation," IEEE Symposium on Security and Privacy, May 19-22, 2013, 16 pages. cited by applicant

Parno, "A Note on the Unsoundness of vnTinyRAM's Snark," retrieved from <https://eprint.iacr.org/2015/437>, May 6, 2015, 4 pages. cited by applicant

Prasad et al., "Effect of Quine-McCuskey Simplification on Boolean Space Complexity," IEEE Xplore, Jul. 25-26, 2009, 6 pages. cited by applicant

Ritzdorf et al., "TLS-N: Non-repudiation over TLS Enabling Ubiquitous Content Signing for Disintermediation," IACR ePrint report, first disclosed 2017 [retrieved May 2, 2018], 16 pages. cited by applicant

Satoshi et al., "Connection Limits," Bitcoin Forum, Aug. 9, 2010, https://bitcointalk.org/index.php?topic=741.0;prev_next=prev, 2 pages. cited by applicant

Schaeffer et al., "ZoKrates—a Toolbox for zkSnarks on Ethereum," <https://github.com/Zokrates/ZoKrates>, Feb. 4, 2019, 3 pages. cited by applicant

Schoenmakers et al., "Trinocchio: Privacy-Preserving Outsourcing by Distributed Verifiable Computation," International Conference on Applied Cryptography and Network Security, Jun. 19, 2016, <https://eprint.iacr.org/2015/480.pdf>, 33 pages. cited by applicant

Spencertruman et al., "[Whitepaper] Witnet: A Decentralized Oracle Network Protocol," Bitcoin Forum, Dec. 11, 2017 [retrieved May 2, 2018], <https://bitcointalk.org/index.php?topic=2567253.0>, 9 pages. cited by applicant

Stuart, "EECS Presents Awards for Outstanding PhD and SM Theses," EECS, Nov. 8, 2017, 2 pages. cited by applicant

Sward et al. "Data Insertion in Bitcoin's Blockchain" [online] Augustana College, Jul. 2017 [retrieved Feb. 10, 2022]. Retrieved from the Internet: URL: <https://digitalcommons.augustana.edu/cgi/viewcontent.cgi?article=1000&context=cscfaculty> 2017, 19 pages. cited by applicant

Tarr et al., "Merkle Tree Logs #27," GitHub Secure-Scuttlebutt project page, Sep. 17, 2014 [retrieved May 10, 2018], <https://github.com/ssbc/secure-scuttlebutt/issues/27>, 4 pages. cited by applicant

Tarr, "Merkle Tree Logs #27," <https://github.com/ssbc/secure-scuttlebutt/issues/27>, Sep. 17, 2014, 4 pages. cited by applicant

Tarr, "Tree-Exchange," Github, Aug. 4, 2014 (last updated Sep. 17, 2014) [retrieved May 10, 2018], <https://github.com/dominictarr/tree-exchange>, 3 pages. cited by applicant

Teutsch et al., "A scalable verification solution for blockchains," Nov. 16, 2017, <https://people.cs.uchicago.edu/~teutsch/papers/truebit.pdf>, 50 pages. cited by applicant

Tillich et al., "Circuits of basic functions suitable for MPC and FHE," <https://homes.esat.kuleuven.be/~nsmart/MPC/>, first disclosed 2012, retrieved May 2, 2018, 2 pages. cited by applicant

Todd, "[bitcoin-dev] Building Blocks of the State Machine Approach to Consensus," [petertodd.org](https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2016-June/012773.html), Jun. 20, 2016, <https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2016-June/012773.html>, six pages. cited by applicant

Turner, "Cse 260. Introduction to Digital Logic and Computer Design," Syllabus and Text Book, <https://www.arl.wustl.edu/~jst/cse/260/ddc.pdf>, 2015, 435 pages. cited by applicant

Turner, "Designing Digital Circuits a modern approach," CSE 260, Introduction to Digital Logic and Computer Design, Spring 2014, 435 pages. cited by applicant

UK Commercial Search Report mailed Dec. 3, 2018, Patent Application No. GB1806444.4, 8 pages. cited by applicant

UK Commercial Search Report mailed Feb. 2, 2018, Patent Application No. GB1718505.9, 7 pages. cited by applicant

UK Commercial Search Report mailed May 31, 2018, Patent Application No. GB1801753.3, 8 pages. cited by applicant

UK Commercial Search Report mailed May 4, 2018, Patent Application No. GB1720946.1, 8 pages. cited by applicant

UK Commercial Search Report mailed Oct. 25, 2018, Patent Application No. GB1805948.5, 9 pages. cited by applicant

UK IPO Search Report mailed Jul. 26, 2018, Patent Application No. GB1801753.3, 5 pages. cited by applicant

UK IPO Search Report mailed Jun. 15, 2018, Patent Application No. GB1720946.1, 4 pages. cited by applicant

UK IPO Search Report mailed Nov. 2, 2018, Patent Application No. GB1805948.5, 4 pages. cited by applicant

Al-Riyami, Sattam, S., "Cryptographic Schemes Based on Elliptic Curve Pairings", Technical Report, RHUL-MA-2002-2, Feb. 2, 2005, 278 pages. cited by applicant

Primary Examiner: Steinle; Andrew J

Attorney, Agent or Firm: Tarolli, Sundheim, Covell & Tummino LLP

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation of U.S. patent application Ser. No. 18/134,276, filed Apr. 13, 2023, entitled “SYSTEM FOR SECURING VERIFICATION KEY FROM ALTERATION AND VERIFYING VALIDITY OF A PROOF OF CORRECTNESS,” which is a continuation of U.S. patent application Ser. No. 16/762,471, now U.S. Pat. No. 11,658,801, filed May 7, 2020, entitled “SYSTEM FOR SECURING VERIFICATION KEY FROM ALTERATION AND VERIFYING VALIDITY OF A PROOF OF CORRECTNESS,” which is a 371 National Stage of International Patent Application No. PCT/IB2018/058433, filed Oct. 29, 2018, which claims priority to United Kingdom Patent Application No. 1806444.4, filed Apr. 20, 2018, United Kingdom Patent Application No. 1805948.5, filed Apr. 10, 2018, United Kingdom Patent Application No. 1801753.3, filed Feb. 2, 2018, United Kingdom Patent Application No. 1720768.9, filed Dec. 13, 2017, United Kingdom Patent Application No. 1719998.5, filed Nov. 30, 2017, and United Kingdom Patent Application No. 1718505.9, filed Nov. 9, 2017, the disclosures of which are incorporated herein by reference in their entirety.

BACKGROUND

(1) This invention relates generally to blockchain technologies, and more particularly to enabling execution of a zero-knowledge protocol on a blockchain by utilizing a locking script to secure a verification key from alteration and verify the validity of a proof of correctness. This invention further utilises cryptographic and mathematical techniques to enforce security in relation to electronic transfers conducted over a blockchain network. The invention is particularly suited, but not limited to, use in smart contract generation and execution.

SUMMARY

(2) In this document, the term ‘blockchain’ may refer to any of several types of electronic, computer-based, distributed ledgers. These include consensus-based blockchain and transaction-chain technologies, permissioned and un-permissioned ledgers, shared ledgers, and variations thereof. The most widely known application of blockchain technology is the Bitcoin ledger, although other blockchain implementations have been proposed and developed. While Bitcoin may be referred to as a useful application of the technology described in the present disclosure, for the purpose of convenience and illustration, Bitcoin is just one of many applications to which the technology described in the present disclosure may be applied. However, it should be noted that the invention is not limited to use with the Bitcoin blockchain; alternative blockchain implementations and protocols, including non-commercial applications, also fall within the scope of the present invention.

(3) A blockchain is a peer-to-peer, electronic ledger which is implemented as a computer-based decentralised, distributed system made up of blocks which in turn may be made up of transactions and other information. In some examples, a “blockchain transaction” refers to an input message encoding a structured collection of field values comprising data and a set of conditions, where fulfilment of the set of conditions is prerequisite for the set of fields to be written to a blockchain data structure. For example, with Bitcoin each transaction is a data structure that encodes the transfer of control of a digital asset between participants in the blockchain system, and includes at least one input and at least one output. In some embodiments, a “digital asset” refers to binary data that is associated with a right to use. Examples of digital assets include Bitcoin, ether, and Litecoins. In some implementations, transferring control of a digital asset can be performed by reassociating at least a portion of a digital asset from a first entity to a second entity. Each block of the blockchain may contain a hash of the previous block to that blocks become chained together to create a permanent, unalterable record of all transactions which have been written to the blockchain since its inception.

(4) In some examples, “stack-based scripting language” refers to a programming language that supports various stack-based or stack-oriented execution models and operations. That is, the stack-based scripting language may utilize a stack. With the stack, values can be pushed onto the top of the stack or popped from the top of the stack. Various operations performed to the stack can result in pushing or popping one or more of the values to or from the top of the stack. For example, an OP_EQUAL operation pops the top two items from the stack, compares them, and pushes a result (e.g., 1 if equal or 0 if unequal) to the top of the stack. Other operations performed to the stack, such as OP_PICK, may allow items to be selected from positions other than the top of the stack. In some scripting languages employed by some of the present embodiments, there may be at least two stacks: a main stack and an alternate stack. Some operations of the scripting language can move items from the top of one stack to the top of another stack. For example, OP_TOALTSTACK, moves a value from the top of the main stack to the top of the alternate stack. It should be noted that a stack-based scripting language, in some cases, may not be limited solely to operation in a strictly last-in-first-out (LIFO) manner. For example, a stack-based scripting language may support operations that copies or moves the n-th item in the stack to the top (e.g., OP_PICK and OP_ROLL respectively, in Bitcoin). Scripts written in a stack-based scripting language may be pushed onto a logical stack that can be implemented using any suitable data structure such as a vector, list, or stack.

(5) In order for a transaction to be written to the blockchain, it must be “validated”. Network nodes (mining nodes) perform work to ensure that each transaction is valid, with invalid transactions rejected from the network. A node can have standards for validity different from other nodes. Because validity in the blockchain is consensus based, a transaction is considered valid if a majority of nodes agree that a transaction is valid. Software clients installed on the nodes perform this validation work on transactions referencing an UTXO in part by executing the UTXO locking and unlocking scripts. If execution of the locking and unlocking scripts evaluates to TRUE and other validation conditions, if applicable, are met, the transaction is validated by the node. The validated transaction is propagated to other network nodes, whereupon a mining node can select to include the transaction in a blockchain. Thus, in order for a transaction to be written to the blockchain, it must be i) validated by the first node that receives the transaction—if the transaction is validated, the node relays it to the other nodes in the network; and ii) added to a new block built by a mining node; and iii) mined, i.e., added to the public ledger of past transactions. The transaction is considered to be confirmed when a sufficient number of blocks is added to the blockchain to make the transaction practically irreversible.

(6) Although blockchain technology is most widely known for the use of cryptocurrency implementation, digital entrepreneurs have begun exploring the use of both the cryptographic security system Bitcoin is based on and the data that can be stored on the Blockchain to implement new systems. It would be highly advantageous if the blockchain could be used for automated tasks and processes which are not limited to the realm of cryptocurrency. Such solutions would be able to harness the benefits of the blockchain (e.g. a permanent, tamper proof records of events, distributed processing etc.) while being more versatile in their applications.

(7) The present disclosure describes technical aspects of one or more blockchain-based computer programs. A blockchain-based computer program may be a machine readable and executable program recorded in a blockchain transaction. The blockchain-based computer program may comprise rules that can process inputs in order to produce results, which can then cause actions to be performed dependent upon those results. One area of current research is the use of blockchain-based computer programs for the implementation of “smart contracts”. Unlike a traditional contract which would be written in natural language, smart contracts may be computer programs designed to automate the execution of the terms of a machine-readable contract or agreement.

(8) Another area of blockchain-related interest is the use of ‘tokens’ (or ‘coloured coins’) to represent and transfer real-world entities via the blockchain. A potentially sensitive or secret item can be represented by the token which has no discernible meaning or value. The token thus serves as an identifier that allows the real-world item to be referenced from the blockchain.

(9) In embodiments, although interaction with specific entities can be encoded at specific steps in the smart contract, the smart contract can otherwise be automatically executed and self-enforced. It is machine readable and executable. In some examples, automatic execution refers to the execution of the smart contract that is successfully performed to enable transfer of the UTXO. Note that in such examples, “an entity” that is able to cause the transfer of the UTXO refers to an entity that is able to create the unlocking script without being required to prove knowledge of some secret. In other

words, the unlocking transaction can be validated without verifying that the source of the data (e.g., an entity that created the unlocking transaction) has access to a cryptographic secret (e.g., private asymmetric key, symmetric key, etc.). Also, in such examples, self-enforcement refers to the validation nodes of the blockchain network being caused to enforce the unlocking transaction according to the constraints. In some examples, “unlocking” a UTXO (also known as “spending the UTXO”) is used in the technical sense, referring to creating an unlocking transaction that references the UTXO and executes as valid.

(10) A blockchain transaction output includes a locking script and information regarding ownership of digital assets such as Bitcoins. The locking script, which may also be referred to as an encumbrance, “locks” the digital assets by specifying conditions that are required to be met in order to transfer the UTXO. For example, a locking script could require that certain data be provided in an unlocking script to unlock the associated digital assets. The locking script is also known as “scriptPubKey” in Bitcoin. A technique for requiring a party to provide data to unlock a digital asset involves embedding a hash of the data inside the locking script.

(11) Thus, it is desirable to provide methods and systems that improve blockchain technology in one or more of these aspects. Such an improved solution has now been devised. Thus, in accordance with the present invention there is provided a method as defined in the appended claims.

(12) Such an improved solution has now been devised.

(13) Thus, in accordance with the present invention there are provided systems and methods as defined in the appended claims.

(14) In accordance with the invention there may be provided a computer-implemented method for a node of a blockchain network, the computer-implemented method comprising: generating a transaction output of a transaction comprising an indication of a digital asset and a locking script that encodes a set of conditions for transferring control of the digital asset, satisfaction of the set of conditions to be determined based at least in part on a verification key and a proof; generating a transaction input of the transaction, the transaction input comprising: an identifier associated with the transaction output; and an unlocking script comprising the proof; and verifying the set of conditions are satisfied based at least in part on the locking script and the unlocking script; and transferring control of the digital asset in response to verifying satisfaction of the set of conditions.

(15) The verification key may comprises a first plurality of elements of a finite field and the proof comprises a second plurality of elements of the finite field.

(16) The elements of the field may be points on an elliptic curve.

(17) The elements of the finite field may be encoded in a compressed format.

(18) Preferably, the method may include a client encoding the transaction input with the verification key and a first digital certificate associated with the client and a worker encoding the transaction input with the proof and a second digital certificate associated with the worker.

(19) The locking script may comprise instructions that, contingent upon a condition of the set of conditions being unsatisfied by the unlocking script, reclaim the digital asset for the provider of the digital asset.

(20) The locking script may further comprise a redeem script, wherein the verification key and the redeem script encodes sufficient information to determine satisfaction of the at least subset of the set of conditions; and the locking script encodes, as one condition the set of conditions, that a hash of the redeem script matches a predetermined value.

(21) The redeem script may be less than or equal to a predetermined maximum size, such as 520 bytes.

(22) The unlocking script may comprise one or more elements of the verification key, and the redeem script may comprise at least some of the remaining elements of the verification. The unlocking script and the redeem script may collectively comprise the verification key.

(23) The locking script, unlocking script, and redeem script of the transaction may be in accordance with a Pay-to-Script-Hash transaction of a Bitcoin-based protocol or variations thereof.

(24) The transaction may be in accordance with a standard transaction of a blockchain-based protocol such as a Bitcoin protocol.

(25) The scripts, such as the locking script and the unlocking script, may be encoded with commands and data in a stack-based scripting language, wherein the commands and the data are placed on a stack in a last-in-first-out order.

(26) The set of conditions may comprise one or more bilinear constraints.

(27) It is also desirable to provide a system, comprising: a processor; and memory including executable instructions that, as a result of execution by the processor, causes the system to perform any of the methods as claimed.

(28) It is also desirable to provide a non-transitory computer-readable storage medium having stored thereon executable instructions that, as a result of execution by one or more processors of a computer system, cause the computer system to at least perform any of the methods as claimed.

(29) The invention can be described as a verification method/system, and/or as a control method/system for controlling the exchange or transfer of a digital asset via a blockchain. In some embodiments, the digital asset is a token or a portion of cryptocurrency. As explained below, the invention can also be described as a secure method/system for new, improved, and advantageous ways of performing operations via a blockchain network or platform.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- (1) These and other aspects of the present invention will be apparent from and elucidated with reference to, the embodiment described herein. An embodiment of the present invention will now be described, by way of example only, and with reference to the accompany drawings, in which:
- (2) FIG. 1 illustrates a blockchain environment in which various embodiments can be implemented;
- (3) FIG. 2 illustrates a computing environment which may be utilized to implement a protocol in accordance with various embodiments;
- (4) FIG. 3 illustrates a diagram of an environment suitable for performance of a verifiable computation;
- (5) FIG. 4 illustrates an example diagram wherein a point on an elliptic curve is represented using a compressed and uncompressed encoding that is suitable for use with a blockchain script such as a locking or unlocking script, in accordance with an embodiment;
- (6) FIG. 5 illustrates a diagram wherein a client supplies the verification key to be used in validation of a proof;
- (7) FIG. 6 illustrates a diagram wherein a worker supplies the verification key to be used in validation of a proof;
- (8) FIG. 7 illustrates a diagram of a process **700** for generating a redeem script in accordance with an embodiment; and
- (9) FIG. 8 illustrates a computing device that can be used to practice at least one embodiment of the present disclosure.

DETAILED DESCRIPTION

- (10) Reference will first be made to FIG. 1, which illustrates an example blockchain network **100** associated with a blockchain in accordance with an embodiment of the present disclosure. In the embodiment, the example blockchain network **100** comprises blockchain nodes that are implemented as peer-to-peer distributed electronic devices, each running an instance of software and/or hardware that performs operations that follow a blockchain protocol that is, at least in part, agreed to among operators of nodes **102**. In some examples, “nodes” refers to peer-to-peer electronic devices that are distributed among the blockchain network. An example of a blockchain protocol is the Bitcoin protocol.
- (11) In some embodiments, the nodes **102** can be comprised of any suitable computing device (e.g., by a server in a data centre, by a client computing device (e.g., a desktop computer, laptop computer, tablet computer, smartphone, etc.), by multiple computing devices in a distributed system of a computing resource service provider, or by any suitable electronic client device such as the computing device **800** of FIG. 8). In some embodiments, the nodes **102** have inputs to receive data messages or objects representative of proposed transactions, such as a transaction **104**. The nodes, in some embodiments, are queryable for information they maintain, such as for information of a state of the transaction **104**.
- (12) As shown in FIG. 1, some of the nodes **102** are communicatively coupled to one or more other of the nodes **102**. Such communicative coupling

can include one or more of wired or wireless communication. The nodes **102** each maintain at least a portion of a “ledger” of all transactions in the blockchain. In this manner, the ledger would be a distributed ledger. A transaction processed by a node that affects the ledger is verifiable by one or more of the other nodes such that the integrity of the ledger is maintained.

(13) As for which nodes **102** can communicate with which other nodes, it can be sufficient that each of the nodes in the example blockchain network **100** are able to communicate with one or more other of the nodes **102** such that a message that is passed between nodes can propagate throughout the example blockchain network **100** (or some significant portion of it), assuming that the message is one that the blockchain protocol indicates should be forwarded. One such message might be the publication of a proposed transaction by one of the nodes **102**, such as node **102A**, which would then propagate along a path such as a path **106**. Another such message might be the publication of a new block proposed for inclusion onto a blockchain.

(14) In an embodiment, at least some of the nodes **102** are mining nodes that perform complex calculations, such as solving cryptographic problems. A mining node that solves the cryptographic problem creates a new block for the blockchain and broadcasts the new block to others of the nodes **102**. The others of the nodes **102** verify the work of the mining node and, upon verification, accept the block into the blockchain (e.g., by adding it to the distributed ledger of the blockchain). In some examples, a block is a group of transactions, often marked with a timestamp and a “fingerprint” (e.g., a hash) of the previous block. In this manner, each block may become linked to a previous block, thereby creating the “chain” that links the blocks in the blockchain. In embodiments, valid blocks are added to the blockchain by a consensus of the nodes **102**. Also in some examples, a blockchain comprises a list of validated blocks.

(15) In an embodiment, at least some of the nodes **102** operate as validating nodes that validate transactions as described in the present disclosure. In some examples, a transaction includes data that provides proof of ownership of a digital asset (e.g., a number of Bitcoins) and conditions for accepting or transferring ownership/control of the digital asset. In some examples, an “unlocking transaction” refers to a blockchain transaction that reassociates (e.g., transferring ownership or control) at least a portion of a digital asset, indicated by an UTXO of a previous transaction, to an entity associated with a blockchain address. In some examples, a “previous transaction” refers to a blockchain transaction that contains the UTXO being referenced by the unlocking transaction. In some embodiments, the transaction includes a “locking script” that encumbers the transaction with conditions that must be fulfilled before ownership/control can be transferred (“unlocked”).

(16) In some embodiments, the blockchain address is a string of alphanumeric characters that is associated with an entity to which control of at least a portion of a digital asset is being transferred/reassociated. In some blockchain protocols implemented in some embodiments, there is a one-to-one correspondence between a public key associated with the entity and the blockchain address. In some embodiments, validation of transactions involves validating one or more conditions specified in a locking script and/or unlocking script. Upon successful validation of the transaction **104**, the validation node adds the transaction **104** to the blockchain and distributes it to the nodes **102**.

(17) Systems and methods described herein relate to enabling a locking script to secure the verification key $V_{sub.K}$ from alteration and checking validity of a proof π , thereby allowing execution of a zero-knowledge protocol on a blockchain during transaction validation.

(18) A verifiable computation is a technique that allows the generation of proofs of computation. In an embodiment, such a technique is utilized by a client to outsource, to another computing entity referred to herein as a worker, the evaluation of a function f on an input x . In some cases, the client is computationally limited so that it is infeasible for the client to perform the evaluation of the function (e.g., the expected runtime of the calculation using computing resources available to the client exceeds a maximum acceptable threshold), although such need not be the case, and the client may, generally, speaking, delegate evaluation of the function f on the input x based on any suitable criterion, such as computational runtime, computational cost (e.g., the financial cost of allocating computing resources to perform the evaluation of the function), and more.

(19) A worker, in an embodiment, is any suitable computing entity such as a blockchain node as described in greater detail elsewhere in the present disclosure. In an embodiment, a worker (e.g., a blockchain node) evaluates the function f on input x and generates an output y and a proof π of the correctness of the output y that can be verified by other computing entities such as the client as described above and/or other nodes of the blockchain network. Proofs, which may also be referred to as arguments, can be verified faster than doing the actual computational—accordingly, computational overhead can be reduced (e.g., reducing power overhead and the cost associated with powering and running computing resources) by verifying the correctness of the proof instead of re-computing the function f over input x to determine the correctness of the output generated by the worker described above. In zero-knowledge verifiable computation the worker provides an attestation to the client that the worker knows an input with a particular property.

(20) An efficient variant of a zero-knowledge proof of knowledge is zk_SNARK (Succinct Non-interactive ARGument of Knowledge). In an embodiment, all pairings-based zk-SNARKs include a process where the worker computes a number of group elements using generic group operations and the verifier checks the proof using a number of pairing product equations. In an embodiment, the linear interactive proof works over a finite field and the worker's and verifier's message include, encode, reference, or otherwise include information usable to determine vectors of field elements.

(21) In an embodiment, systems and methods described herein allow mining nodes of a blockchain to perform a computation (e.g., evaluation of function f on input x) once and generate a proof that can be used to verify correctness of the output wherein evaluating correctness of the proof is computationally less expensive than evaluating the function. In this context, the cost (i.e., how expensive) of operations and tasks may refer to the computational complexity of performing the operation or task. In an embodiment, computational complexity refers to the average computational cost or the worst-case computational cost of performing the sorting algorithm—for example, a heapsort algorithm and a quicksort algorithm both have an average computational cost of $O(n \log n)$, but quicksort has a worst-case computational cost of $O(n \cdot \sup{2})$ whereas heapsort has a worst-case computation cost of $O(n \log n)$. In an embodiment, the average computational cost and/or the worst-case computational cost of evaluating the function f on input x is worse than that of evaluating correctness of the proof. Accordingly, the use of systems and methods described herein are highly advantageous and, may, for example, allow for more computationally expensive contracts to be run as such contracts would not increase the time required to validate the blockchain proportionally. Further advantages may include reduction in power consumption of verifier systems, thereby improving the efficiency of verifier computer systems and reducing the energy costs associated with running such verifier computer systems in evaluating correctness of proofs. Currently, smart contracts must be executed and validated on all nodes—this constraint limits the complexity of smart contracts. Methods and systems described herein can be utilized to implement a system that improves the efficiency of the blockchain by executing a contract once to generate a proof of correctness and, based on the proof of correctness provided by the worker and the verification key provided by the client, all nodes of the blockchain can verify validity of the contract. In this way, the efficiency of the blockchain is improved by increasing the throughput of smart contracts that can be performed in aggregate by nodes of the blockchain and/or enable the computation of more computationally expensive smart contracts.

(22) In an embodiment, a verification key $V_{sub.K}$ or portions thereof can be extracted from public parameters generated in a setup phase of a zero-knowledge protocol and used together with a proof π , and the input/output data to verify the alleged proof of correctness computation provided by a worker. For example, as described in greater detail above and below, systems and methods that allow a locking script secures the verification key $V_{sub.K}$ from alteration and checks the validity of the proof π , allowing the execution of a zero-knowledge protocol on blockchain during transaction validation. Accordingly, the present disclosure presents systems and methods to execute the verification phase using blockchain scripts (e.g., in a Bitcoin-based network) for storing the elements used in the verification of the computation.

(23) FIG. 2 illustrates a computing environment **200** which may be utilized to implement a protocol in accordance with various embodiments. The protocol may be implemented using blockchain technologies to store proof-of-correctness and combine the “correct-by-construction” cryptography approach with smart contracts. In an embodiment, a public verifiable computation scheme comprises three phases: a setup phase, a computation phase, and a verification phase.

(24) A setup phase may be performed as part of a process to outsource some of computational tasks. A client, as referred to below, may refer to an entity such as a customer or client computer system that delegates performance of a computational task to a worker, which may be a different computer system. Clients may, generally speaking, delegate the performance of computational tasks for a variety of reasons, including but not limited to limited computing resources, lack of computing resources, financial costs associated with utilizing a client computer system to perform the task, energy costs associated with utilizing a client computer system to perform the task (e.g., a mobile device or laptop that relies on a battery for power may utilize a worker to perform computationally intensive tasks, thereby saving power and prolonging the usage of the battery-powered device), and more.

(25) In an embodiment, the setup phase involves a client, customer, employee of an organization, or any other suitable entity writing contracts in a formal language with precise semantics. The contract may be written in a high-level programming language such as C or Java. Generally speaking, contracts may be expressed in any language or syntax that is or can be converted to a format that can be manipulated by a computer system. In an embodiment, a domain specific language, with a limited purpose, may provide type-safety and restricted expressivity may be utilized. The source code generated may be a precise description of a contract.

(26) The compiler **202** may be any hardware, software, or a combination thereof that includes executable code that, if executed by one or more processors of a computer system, causes the system to take, as input, the source code **206** and produces a circuit. A compiler **202** may refer to a computer program that executes or performs instructions based on instructions that have been compiled into a machine-readable format such as binary code. It should be noted that while a compiler **202** is illustrated, interpreters, assemblers, and other suitable software and/or hardware component may be utilized to convert the source code to a circuit. In an embodiment, the circuit is an arithmetic circuit that comprises wires that carry values from a field F and connect to logical and/or arithmetic gates. In an embodiment, the circuit  custom character is used by the system to generate a quadratic program Q **208** that comprises a set of polynomials that provide a complete description of the original circuit  custom character.

(27) In an embodiment, the compiler **202** is able to recognise a substantial subset of a programming language such as C or Java including but not limited to: pre-processor directives, static initializers, global and local functions, block-scoped variables, arrays, data structures, pointers, function calls, function operators (e.g., functors), conditionals and loops, and arithmetic and bitwise Boolean operators. In an embodiment, the compiler **202** but does not support the entire set of commands in accordance with standards of the programming language (this may, in some cases, be intended to prevent certain types of algorithms from being executed in a smart contract, such as to prohibit recursive algorithms). In an embodiment, the compiler expands expressions of the source code into an arithmetic gate language to produce an arithmetic circuit. Circuit implementations have been contemplated in the past by Campanelli, M., et al. (2017) in “Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services” and by Tillich, S. and Smart, B in “Circuits of Basic Functions Suitable For MPC and FHE.” The arithmetic circuit may be utilized to build a Quadratic Arithmetic Problem (QAP) by the compiler **202** or any other suitable hardware, software, or combination thereof (e.g., a software module not illustrated in FIG. 2). The quadratic program is compiled into a set of cryptographic routines for the client (e.g., key generation and verification) and the worker (e.g., computation and proof generation) in accordance with an embodiment. In some embodiments, arithmetic circuit optimisation techniques such as those described in U.K. Pat. Application No. 1718505.9 may be utilized in order to reduce the required resources necessary for a worker to determine an outcome of the smart contract.

(28) In an embodiment, the key generator **204** is hardware, software, or a combination thereof that includes executable code which, if executed by one or more processors of a computer system, causes the system to generate an evaluation key and a verification key form a quadratic program. Techniques for encoding computations as quadratic programs are contemplated in “Quadratic Span Programs and Succinct NIZKs without PCs” by Gennaro, R., et al. (2013). In an embodiment, the quadratic arithmetic problem (QAP) Q encodes the circuit  custom character over a field F and contains a set of $m+1$ polynomials:

(29) $V = \{v_k(x)\}, W = \{w_k(x)\}, Y = \{y_k(x)\}$

(30) with $0 \leq k \leq m$. A target polynomial $t(x)$ is also defined. Given a function f that takes n elements of F as input and outputs n' elements, with $N=n+n'$, then Q computes $\text{fif } \{c.\text{sub}.1, \dots, c.\text{sub}.N\} \in F.\text{sup}.N$ is a valid assignment of the group of input and outputs of f and if there exists a list of coefficients $\{c.\text{sub}.N+1, \dots, c.\text{sub}.m\}$ such that $t(x)$ divides $p(x)$:

(31) $p(x) = (v_0(x) + \sum_{k=1}^m \text{Math}.c_k \cdot \text{Math}.v_k(x)) \cdot \text{Math}.(w_0(x) + \sum_{k=1}^m \text{Math}.c_k \cdot \text{Math}.w_k(x)) - (y_0(x) + \sum_{k=1}^m \text{Math}.c_k \cdot \text{Math}.y_k(x))$

(32) Therefore, in an embodiment, there must exist some polynomial $h(x)$ such that $h(x) \cdot \text{Math}.t(x) = p(x)$. The size of Q is m , and its degree is the degree of $t(x)$.

(33) In an embodiment, building a QAP for an arithmetic circuit comprises pick an arbitrary root $r.\text{sub}.g \in F$ for each multiplication gate g in the circuit and defining the target polynomial to be $t(x) = \prod.\text{sub}.g(x - r.\text{sub}.g)$. In an embodiment, an index $k \in \{1 \dots m\}$ is associated to each input of the circuit and to each output from a multiplication gate. The polynomials in V encode the left input into each gate, the W encode the right input into each gate, and the Y encode the outputs. For instance, $v.\text{sub}.k(r.\text{sub}.g) = 1$ if the k -th wire is a left input to gate g , and $v.\text{sub}.k(r.\text{sub}.g) = 0$ otherwise. Therefore, for a particular gate g and its root $r.\text{sub}.g$, the previous Equation can be simplified as follows:

(34) $(\sum_{k=1}^m \text{Math}.c_k \cdot \text{Math}.v_k(r_g)) \cdot \text{Math}.(\sum_{k=1}^m \text{Math}.c_k \cdot \text{Math}.w_k(r_g)) = (\sum_{k \in I_{\text{left}}} \text{Math}.c_k) \cdot \text{Math}.(\sum_{k \in I_{\text{right}}} \text{Math}.c_k) = c_g y_k(r_g) = c_g$

(35) The output value of the gate is equal to the product of its inputs. The divisibility check decomposes into $\deg(t(x))$ separate checks, one for each gate g and root $r.\text{sub}.g$ of $t(x)$, such that $p(r.\text{sub}.g) = 0$. Addition gates and multiplication-by-constant gates do not contribute to the size or degree of the QAP.

(36) In an embodiment, the QAP is defined over a field F_p , where p is a large prime. In an embodiment, it is desirable that QAP over F_p efficiently computes any function that can be expressed in terms of addition and multiplication modulo p . An arithmetic split gate may be designed to translate an arithmetic wire $a \in F_p$, known to be in $[0, 2.\text{sup}.k-1]$, into k binary output wires. Accordingly, it follows that, Boolean functions can be expressed using arithmetic gates.

(37) For instance, $\text{NAND}(a,b) = 1 - ab$. Each embedded Boolean gate costs only one multiply. Moreover, new gates such as split can be defined as standalone and composed with other gates. Given input $a \in F.\text{sub}.p$ known to be in $[0, 2.\text{sup}.k-1]$, the split gate outputs k wires holding the binary digits $a.\text{sub}.1, \dots, a.\text{sub}.k$ of a such $\sum.\text{sup}.k 2.\text{sup}.i-1 a.\text{sub}.i = a$ and each $a.\text{sub}.i$ is either 0 or 1.

(38) Finally, the public parameters to be used by all provers and verifiers are generated by the system as part of the setup phase. It should be noted that the evaluation key $E.\text{sub}.K$ and the verification key $V.\text{sub}.K$ are derived using a secret value selected by the client. A key generator **204** may utilize the quadratic arithmetic program (QAP) in connection with the key generation algorithm to generate the evaluation key $E.\text{sub}.K$ **210** and the verification key $V.\text{sub}.K$ **212**.

(39) In an embodiment, performing a computational task involves the computation of a function on an input **216** (i.e., a process for evaluating $f(x)$) by a worker. In an embodiment, the worker is any suitable computer system that the client may delegate a computational task to. The input **216**, in an embodiment, includes information that attests to the worker's identity, such as a digital signature generated using a private key associated with the worker. In an embodiment, the worker is a computer system that the client pays a fee for a successful computation (e.g., via a transfer of digital assets). The client, in an embodiment provides an input x and the evaluation key $E.\text{sub}.K$ to a worker, the worker uses the evaluation module **214** to a compute routine to compute the output y (i.e., $y=f(x)$) wherein the input is x and the function is f) and uses the evaluation key $E.\text{sub}.K$ **210** to produce a proof-of-correctness **218**. The evaluation module, in an embodiment, is hardware and/or software that includes instructions that, if executed by one or more processors of a computer system, cause the computer system to evaluate the values of the internal circuit wires of the QAP **208** and produce an output y of the QAP.

(40) In an embodiment, each verification $v_{\text{sub.k}(x)} \in F$ of the quadratic program is mapped to an element $g_{\text{sup.vk}(s)}$ in a bilinear group, where s is a secret value selected by the client, g is a generator of the group, and F is the field of discrete logarithms of g . In an embodiment, for a given input, the worker evaluates the circuit to obtain the output and the values of the internal circuit wires, which correspond to the coefficients $c_{\text{sub.i}}$ of the quadratic program. Accordingly, a worker can evaluate $v(s) = \sum_{\text{sub.k} \in \{m\}} c_{\text{sub.k}} \cdot \text{Math.v.sub.k}(s)$ to get $g_{\text{sup.v}(s)}$; compute $w(s)$ and $y(s)$; compute $h(x) = p(x)/t(x) = \sum_{\text{sup.dh.sub.i}} \text{Math.x.sub.i}$; and compute $g_{\text{sup.h}(s)}$ using the $h_{\text{sub.i}}$ and $g_{\text{sup.s}(i)}$ terms in the evaluation key. In an embodiment, the proof-of-correctness **218** comprises $(g_{\text{sup.v}(s)}, g_{\text{sup.w}(s)}, g_{\text{sup.y}(s)}, g_{\text{sup.h}(s)})$ and a verifier uses the bilinear map to check that $p(s) = h(s) \cdot \text{Math.t}(s)$. In an embodiment, the proof π is stored on the blockchain **222** for later use or can be verified by multiple parties without requiring the prover to separately interact with each of these. In an embodiment, the evaluation of the circuit storage of the proof-of-correctness may be performed to unlock digital assets encumbered by a locking script of a transaction.

(41) In an embodiment, the proof π is broadcast to a blockchain network and a verifier **220** is used to verify the proof. In an embodiment, the verifier **220** is any suitable computing entity, such as a node on a blockchain. It should further be noted that in some cases, the same computing entity that generates the evaluation key $E_{\text{sub.K}}$ and verification key $V_{\text{sub.K}}$ also verifies the proof. In an embodiment, nodes of blockchain can validate a payment transaction using the verification key $V_{\text{sub.K}}$ and the proof π , thus validating the contract if the verification succeeds. One requirement of the protocol is that the worker cannot provide incorrect proofs, even when it knows the verification key $V_{\text{sub.K}}$. Thus, in this protocol, a common reference string (CRS) is produced by the client or by a trusted third party who publishes at least the evaluation key $E_{\text{sub.K}}$ and verification key $V_{\text{sub.K}}$. In an embodiment, the published verification key $V_{\text{sub.K}}$ can be used by any computing entity to verify computations.

(42) Using techniques described herein, a client is able to partially obfuscate transaction data, such as the identity of the recipients of a blockchain transaction. In an embodiment, the unlocking script does not expose the recipient's address and the recipient's public key. However, in some cases, the value of the transaction (e.g., amount of digital assets transferred) may be visible to nodes of the blockchain network. In an embodiment, cryptographic techniques as described above and below are utilized by the client to covert locking scripts into quadratic arithmetic programs and the worker to solve arithmetic programs to generate proofs.

(43) Generally speaking, a client is able to use standard transactions (e.g., standard transactions as defined in a Bitcoin-based blockchain network) such as P2PK and P2PKH to pay a counterparty or worker. For example, in an embodiment, a client converts a P2PK locking script into an arithmetic circuit and broadcasts a payment transaction that includes a puzzle derived from the circuit. A counterparty or worker receives the circuit, provides an appropriate input (e.g., information that attests to the worker's identity such as a shared secret between the client and the worker or a digital signature generated using the worker's private key) and runs the circuit to generate a proof-of-correctness π . In an embodiment, the proof is used to unlock digital assets, and furthermore, it may be the case that information identifying the counterparty or worker (e.g., a public key and/or digital signature associated with the counterparty or worker) is not recorded to the blockchain in an unobfuscated format.

(44) In an embodiment, the verification key and the corresponding proof are generated according to techniques described above and/or below. Accordingly, a verifier is given verification key $V_{\text{sub.K}}$ and proof π :

$$(45) V_K = \left\{ \begin{array}{l} Q \\ v_Q \\ w_Q \\ w \\ y_Q \\ \beta \end{array} \right\} \quad \text{Proof} = \left\{ \begin{array}{l} \text{Math.}_{i=N+1}^m a_i r_v v_i(s) \\ \text{Math.}_{i=N+1}^m a_i v r_v v_i(s) \\ \text{Math.}_{i=N+1}^m a_i r_w w_i(s) Q \\ \text{Math.}_{i=N+1}^m a_i w r_w w_i(s) \\ \text{Math.}_{i=N+1}^m a_i r_y y_i(s) \\ \text{Math.}_{i=N+1}^m a_i y r_y y_i(s) \\ \text{Math.}_{i=N+1}^m a_i (r_v v_i(s) + r_w w_i(s) + r_y y_i(s)) \\ \text{Math.}_{i=0}^d h_i s^i Q \end{array} \right\}$$

such that the verifier computes a plurality of elliptic curve multiplications (e.g., one for each public input variable) and five pair checks, one of which includes an additional pairing multiplication.

(46) Given verification key $V_{\text{sub.K}}$, proof π , and $(a_{\text{sub.1}}, a_{\text{sub.2}}, \dots, a_{\text{sub.N}})$, to verify that $t(x)$ divides $p(x)$ and hence $(x_{\text{sub.N+1}}, \dots, x_{\text{sub.m}}) = f(x_{\text{sub.0}}, \dots, x_{\text{sub.N}})$, the verifier proceeds as follows. First it checks all the three α terms:

$$e(\alpha_{\text{sub.vr.sub.vV.sub.mid}(s)}, \alpha_{\text{sub.vV.sub.mid}(s)}) = e(r_{\text{sub.vV.sub.mid}(s)}, \alpha_{\text{sub.vV.sub.mid}(s)}) \\ e(\alpha_{\text{sub.wr.sub.wW.sub.mid}(s)}, \alpha_{\text{sub.wW.sub.mid}(s)}) = e(r_{\text{sub.wW.sub.mid}(s)}, \alpha_{\text{sub.wW.sub.mid}(s)}) \\ e(\alpha_{\text{sub.yr.sub.yY.sub.mid}(s)}, \alpha_{\text{sub.yY.sub.mid}(s)}) = e(r_{\text{sub.yY.sub.mid}(s)}, \alpha_{\text{sub.yY.sub.mid}(s)})$$

wherein $V_{\text{sub.mid}(s)} = \sum_{\text{sub.i=N+1}}^{\text{sup.ma.sub.iv.sub.i}(s)}$, $W_{\text{sub.mid}(s)} = \sum_{\text{sub.i=N+1}}^{\text{sup.ma.sub.iw.sub.i}(s)}$, and $Y_{\text{sub.mid}(s)} = \sum_{\text{sub.i=N+1}}^{\text{sup.ma.sub.iy.sub.i}(s)}$. Then, the verifier checks the term β :

$$(47) e(r_v V_{\text{mid}(s)} + r_y Y_{\text{mid}(s)}, Q) \cdot \text{Math.} e(\beta, r_w W_{\text{mid}(s)} Q) = e(Z_{\text{mid}(s)}, Q)$$

and $Z_{\text{sub.mid}(s)} = \sum_{\text{sub.i=N+1}}^{\text{sup.ma.sub.i}(r_{\text{sub.v}} v_{\text{sub.i}(s)} + r_{\text{sub.w}} w_{\text{sub.i}(s)} + r_{\text{sub.y}} y_{\text{sub.i}(s)})$. Finally, the verifier checks the divisibility requirement:

$$(48) e(r_v V(s), r_w W(s) Q) = e(r_y Y(s), Q) \cdot \text{Math.} e(r_y t(s), h(s) Q)$$

wherein $r_{\text{sub.vV}(s)} = \sum_{\text{sub.i=0}}^{\text{sup.mr.sub.va.sub.iv.sub.i}(s)}$, $r_{\text{sub.wW}(s)} = \sum_{\text{sub.i=0}}^{\text{sup.mr.sub.wa.sub.iw.sub.i}(s)}$, $r_{\text{sub.yY}(s)} = \sum_{\text{sub.i=0}}^{\text{sup.mr.sub.ya.sub.iy.sub.i}(s)}$, and $h(s) = \sum_{\text{sub.i=0}}^{\text{sup.dh.sub.i}}$.

(49) Thus, upon considering the notation from the sections described above and the examples described in this disclosure, the verification comprises a set of pair checks of the following elements, in accordance with one embodiment:

$$(50) e(\binom{2}{2}, V_K^2) = e(\binom{1}{1}, V_K^3) \cdot e(\binom{4}{4}, V_K^2) = e(V_K^5, \binom{3}{3}) \cdot e(\binom{6}{6}, V_K^2) = e(\binom{5}{5}, V_K^6) \cdot e(\binom{1}{1} + \binom{6}{6}, V_K^2) = e(\binom{7}{7}, V_K^2)$$

$$e((a_0 V_K^{10} + a_1 V_K^{11} + a_2 V_K^{12} + a_3 V_K^{13} + a_4 V_K^{14} + a_5 V_K^{15} + a_6 V_K^{16} + a_7 V_K^{17}), (a_0 V_K^{16} + a_1 V_K^{17} + a_2 V_K^{18} + a_3 V_K^{19} + a_4 V_K^{20} + a_5 V_K^{21} + a_6 V_K^{22} + a_7 V_K^{23} + a_8 V_K^{24})) = e((a_0 V_K^{22} + a_1 V_K^{23} + a_2 V_K^{24} + a_3 V_K^{25} + a_4 V_K^{26} + a_5 V_K^{27} + a_6 V_K^{28} + a_7 V_K^{29} + a_8 V_K^{30}), (a_0 V_K^{30} + a_1 V_K^{31} + a_2 V_K^{32} + a_3 V_K^{33} + a_4 V_K^{34} + a_5 V_K^{35} + a_6 V_K^{36} + a_7 V_K^{37} + a_8 V_K^{38}))$$

(51) FIG. 3 illustrates a diagram **300** for coordinating the performance of a verifiable computation. The client **302**, worker **304**, and verifier **306** may be nodes of a blockchain network. The client **302** may be any suitable computer system any may include executable code which, if executed by one or more processors of a computer system, causes the computer system to receive a smart contract **308**. In an embodiment, the smart contract **308** is encoded in a high-level programming language as source code such as C, C++, or Java. In an embodiment, software such as a compiler, interpreter, and/or assembler may be utilized to transform the smart contract **308** to an arithmetic circuit **310** which consists of "wires" that carry values from a field F and connect to addition and multiplication gates. It should be noted that the arithmetic circuit may refer to a logical circuit that can be implemented by a physical circuit comprising a series of physical gates (e.g., using transistor-transistor logic (TTL) integrated circuits such as 7400-series gates, flip-flops, buffers, decoders, multiplexers, and the like) connected by physical wires.

(52) In an embodiment, the client **302** provides the worker **304** with an arithmetic circuit **310** and an input **312** to the circuit. The circuit **310** may be used to generate a quadratic program Q that comprises a set of polynomials that provide a complete description of the original circuit. In either case,

the worker **304** may execute the circuit **400** of the quadratic program **402** to generate one or more outputs **314**. In some embodiments, the worker (i.e., the prover) is expected to obtain, as the output, a valid transcript for {**402** custom character, **x**, **402** custom character} that is an assignment of values to the circuit wires such that the values assigned to the input wires are those of **x**, the intermediate values correspond to the correct operation of each gate in **402** custom character, and the values assigned to the output wire(s) is **402** custom character; if the claimed output is incorrect (i.e., **402** custom character $\neq$ **402** custom character(**x**)), then a valid transcript for {**402** custom character, **x**, **402** custom character} does not exist. In an embodiment, the worker is expected to provide a subset of the values of the circuit wires, wherein the selected subset of the values of the circuit wires are not known to the worker a priori.

(53) In embodiments, the output **402** custom character, the values of the internal circuit wires (or a subset thereof), and the evaluation key EKE.sub.K are used to produce the proof-of-correctness **316**. The proof π can be stored on the blockchain and verified by multiple parties without requiring the worker **304** to separately interact with the multiple parties. In this manner, a verifier **306** can validate the payment transaction using the public verification key V.sub.K and the proof π , thereby validating the contract. In some cases, the client **302** may reclaim digital assets encumbered by the payment transaction if the verification fails. In some cases, the verifier **306** and the client **302** are the same computer system.

(54) FIG. 4 illustrates a diagram **400** of an embodiment of the present disclosure. Specifically, FIG. 4 depicts a point on an elliptic curve **402** that is encoded in various formats that are suitable for use in a blockchain based script such as a Bitcoin-based locking and unlocking script.

(55) In various embodiments, elliptic curve points may be encoded in locking and unlocking scripts that are executed in association with transactions. On a Bitcoin-based system, these scripts may be written in a stack-based scripting language. For example, the verification key V.sub.K may comprise {V.sub.K.sup.1, V.sub.K.sup.2, V.sub.K.sup.3, . . . , V.sub.K.sup.n} and the proof π may comprise a set of elements { π .sub.1, . . . , π .sub.8} wherein V.sub.K.sup.i and π .sub.j are points on an elliptic curve over a finite field **402** custom character.sub.p, E(**402** custom character.sub.p). FIG. 4 illustrates an example diagram **400** wherein a point on an elliptic curve **402** is represented using either a compressed (P.sub.C) or an uncompressed (P.sub.U) encoding that is suitable for use with a blockchain script such as a locking or unlocking script.

(56) In an embodiment, let $P \in E(\text{402 custom character.sub.p})$ be a point on the elliptic curve. If $P \neq \text{402 custom character}$, the point is represented by its affine coordinate. In uncompressed encoding **404** the point P is represented by information indicating an uncompressed encoding (e.g., a prefix **406** such as the parameter 'C' described below) and two field elements (x and y coordinates **408** and **410**), while in the compressed encoding the point is represented only by its x-coordinate and an additional bit to uniquely identify the y-coordinate. Accordingly, in an embodiment, $P_{\text{sub.U}} = C \| X \| Y$ where: $C = 0x04$ $X = \text{FieldElements2OctetString}(x)$ $Y = \text{FieldElements2OctetString}(y)$ wherein the double-pipe operator " $\|$ " refer to a concatenation operation and the FieldElements2OctetString() function can be used to convert elements of fields (e.g., finite fields) to octet strings comprising exactly eight bits. Regarding the compressed encoding, $P_{\text{sub.C}} = C \| X$ where:

(57)
$$C = \begin{cases} 0 \times 02 & \text{if even} \\ 0 \times 03 & \text{if odd} \end{cases} \quad X = \text{FieldElements2OctetString}(x)$$

(58) In an embodiment, an uncompressed encoding **404** comprises a point P of a finite field (i.e., $P \in E(\text{402 custom character.sub.p})$) such as the x-coordinate **408** and y-coordinate **410** of a point on an elliptic curve.

(59) The uncompressed encoding **404** further comprises information usable to determine the encoding is an uncompressed encoding (e.g., a prefix **406** value prepended/appended to the x-coordinate **408** and the y-coordinate **410**). Conversely, a compressed encoding **412** comprises an encoding of a point P of a finite field (e.g., as described above) and information usable to determine the encoding is a compressed encoding. For example, a compressed encoding **412A** encodes a prefix **414** and the x-coordinate **408** of a point P, wherein the corresponding y-coordinate is determinable based at least in part on the prefix **414**. For example, the prefix **414** indicates the y-coordinate is even. Conversely, a second compressed encoding **412B** encodes a different prefix **416** and an x-coordinate **408** of a point P, and is used to determine a y-coordinate that is odd.

(60) It should be noted that, generally speaking, the uncompressed point P.sub.U may be expressed in any suitable format that encodes information usable to determine that P.sub.U is an uncompressed elliptic curve point, the x-coordinate of the point, and the y-coordinate of the point. Similarly, the compressed point P.sub.C may be expressed in any suitable format that encodes information usable to determine that P.sub.C is a compressed elliptic curve point and compressed information that is usable to determine the x-coordinate and y-coordinate of the point.

(61) As an example, consider the representation of points of a secp256k1 elliptic curve in a Bitcoin-based system. In an embodiment where uncompressed points are represented, the first octet in an encoded bit string of an uncompressed elliptic curve P is 0x04 followed by two 256-bit numbers corresponding to the X coordinate of the point and the Y coordinate of the point (i.e., $P = C \| X \| Y$). As an example, a Distinguished Encoding Rules (DER) encoding format is used, although other suitable encoding formats are contemplated in the scope of this disclosure such as Basic Encoding Rules (BER) and Canonical Encoding Rules (CER).

(62) In an embodiment, a one-byte script opcode including the length of the elliptic curve point is concatenated to the actual point (e.g., the length is concatenated to the front of the actual point). For example, if: $x = 0xe3b01684a8a8b66f8e44203db5869b4dcb74a0afc905ae9197ed74a8d6cecdcc$ $y = 0x6424d186a23687532c8b20911defc2f42c93749b3736857912c6abe2dc3f01d1$

the compressed and uncompressed scripts P.sub.C and P.sub.U are, in an embodiment, respectively: Script P.sub.C: 0x21

0x03e3b01684a8a8b66f8e44203db5869b4dcb74a0afc905ae9197ed74a8d6cecdcc Script P.sub.U: 0x41

0x04e3b01684a8a8b66f8e44203db5869b4dcb74a0afc905ae9197ed74a8d6cecdcc6424d1

86a23687532c8b20911defc2f42c93749b3736857912c6abe2dc3f01d1

(63) In an embodiment, a client is a party of a smart contract and determines to utilize a blockchain for execution of the contract. As part of the determination, the client posts a payment transition to the blockchain to transfer digital assets controlled by the client to a party (e.g., a worker) that publishes a proof π . A verifier compute system such as those described above may perform a verification routine to determine that the proof π is correct.

(64) In accordance with a protocol, the client may generate a common reference string for the production and generation of the proof π . In some embodiments, the common reference string is generated by a trusted party (e.g., a third party). It should be noted that while a trusted third party may generate the common reference string, that the introduction of a trusted third party to the protocol is optional—the protocol requires a client and worker and may be performed without an additional third party.

(65) Continuing with the protocol, the verification of the computation occurs as part of the verification stage of a transaction. In an embodiment, such as a Bitcoin-based network, the client or worker is constrained to provide {V.sub.K.sup.1, V.sub.K.sup.2, V.sub.K.sup.3, . . . , V.sub.K.sup.n} as part of a locking script and/or an unlocking script. This constraint can be achieved in various ways. For example, in an embodiment, the worker provides the verification key V.sub.K through the injection of a serialised unlocking transaction that comprises {V.sub.K.sup.1, V.sub.K.sup.2, V.sub.K.sup.3, . . . , V.sub.K.sup.n} in one of its unlocking scripts. As a second example, the client broadcasts a transaction where the locking script comprises {V.sub.K.sup.1, V.sub.K.sup.2, V.sub.K.sup.3, . . . , V.sub.K.sup.n} and equations utilized in the Validation phase and creates a transaction input signed with **SIGHASH_NONE**|**SIGHASH_ANYONECANPAY** that comprises verification key V.sub.K. The client passes this input to the worker, who needs to add { π .sub.1, . . . , π .sub.8}, sign, and broadcast. As yet another example, the worker provides verification key V.sub.K by using a fixed-length script hash (e.g., 20-byte script hash) in the locking script of the transaction paying for the execution. Of course, these are merely illustrative examples of how {V.sub.K.sup.1, V.sub.K.sup.2, V.sub.K.sup.3, . . . , V.sub.K.sup.n} are provided as part of a locking script, an unlocking script, or any other suitable scripts or operations that are executed as part of the verification of a transaction in accordance with any suitable blockchain protocol. In an embodiment, the worker provides { π .sub.1, . . . , π .sub.8} as part of an unlocking script and the proof π is authorised if it meets the conditions set in the output script.

(66) A blockchain-based system such as a Bitcoin-based system, supports execution of operations (also referred to as opcodes and commands) in accordance with a scripting language such as Script (e.g., Bitcoin-based systems support Script). In an embodiment, a transaction comprises locking (output) and unlocking (input) scripts and can be identifiable as one of a list of standard types. For example, in a Bitcoin-based system, there are five standard types: Pay-to-Public-Key (P2PK), Pay-to-Public-Key-Hash (P2PKH), Multi-signature, Pay-to-Script-Hash (P2SH), and OP_RETURN.

(67) In running the proposed zk-protocol as described herein, information such as the common reference string, the proof π , and/or portions thereof are stored on the blockchain. Furthermore, as part a validation process, it may also be the case that a system is to extract specific elements pushed on the stack in the transaction validation process and check the pairing from the validation phase as described above.

(68) Transactions may include small programs known as scripts embedded into their inputs and outputs, which specify how and by whom the outputs of the transactions can be accessed. On the Bitcoin platform, these scripts are written using a stack-based scripting language. In an embodiment, a protocol includes various technical rules and syntax-based restrictions on transactions that determine whether a transaction is a standard transaction or a non-standard transaction. In an embodiment, there are restrictions to the size of elements pushed on the execution stack and on the total size of input scripts. For example, in a Bitcoin-based system, every element pushed on the execution stack is limited to 520 bytes and each input script is limited to 1650 bytes. In an embodiment, after the script execution, the stack includes exactly one non-false element. Input scripts cannot include any OP codes other than OP_PUSHDATA (except for the redeem script portion). In an embodiment, a Bitcoin-based system requires a minimum output value of 546 satoshis, although it should be noted that different blockchain systems may define a different amount and/or unit of digital assets to be included as an output value (e.g., a minimum payment to the mining nodes of the transaction). In an embodiment, the minimum output value is zero (i.e., there is no required transferred of digital assets). Transactions that deviate from these rules are considered non-standard.

(69) As part of the validation phase, a verifier (e.g., the client) extracts elements of the verification key $V_{\text{sub.K}}$ and proof π and uses them in pairing checks. In an embodiment, the verification check comprises a bilinear mapping that satisfies a set of constraints. As an example, consider a bilinear mapping $e(x, y) = 2^{\text{sup.xy}}$ that satisfies constraints of the form $e(V_{\text{sub.K}}^{\text{sup.1}}, \pi_{\text{sub.1}} + \pi_{\text{sub.2}}) = e(V_{\text{sub.K}}^{\text{sup.1}}, \pi_{\text{sub.1}}) * e(V_{\text{sub.K}}^{\text{sup.1}}, \pi_{\text{sub.2}})$, such as in the following manner:

$$(70) e(3, 4 + 5) = 2^{3*9} = 2^{27} e(3, 4) * e(3, 5) = 2^{3*4} 2^{3*5} = 2^{12} 2^{15} = 2^{27}$$

(71) It should be noted that such a pairing described above is for illustrative purposes—while such a mapping may not be useful, verbatim, for application in cryptographic systems, it should be noted that the process for verifying the bilinear constraints remains the same wherein the bilinear maps are extended over elliptic curves. Again, for illustrative purposes, an example for how to extract and use information from the common reference string (e.g., the verification key $V_{\text{sub.K}}$) and proof π in the validation phase using Bitcoin scripts is shown below:

(72) TABLE-US-00001 Verification Key $V_{\text{sub.K}} = \{V_{\text{sub.K}}^{\text{sup.1}}, V_{\text{sub.K}}^{\text{sup.2}}, V_{\text{sub.K}}^{\text{sup.3}}, V_{\text{sub.K}}^{\text{sup.4}}, V_{\text{sub.K}}^{\text{sup.5}}, V_{\text{sub.K}}^{\text{sup.6}}\}$ Proof $\pi = \{\pi_{\text{sub.1}}, \pi_{\text{sub.2}}, \pi_{\text{sub.3}}, \dots, \pi_{\text{sub.8}}\}$ Validation check $e(\pi_{\text{sub.2}}, V_{\text{sub.K}}^{\text{sup.2}}) = e(\pi_{\text{sub.1}}, V_{\text{sub.K}}^{\text{sup.3}}) e(\pi_{\text{sub.4}}, V_{\text{sub.K}}^{\text{sup.2}}) = e(V_{\text{sub.K}}^{\text{sup.5}}, \pi_{\text{sub.3}}) e(\pi_{\text{sub.6}}, V_{\text{sub.K}}^{\text{sup.2}}) = e(\pi_{\text{sub.5}}, V_{\text{sub.K}}^{\text{sup.6}}) \dots$

(73) It should be noted that in the example provided above is merely used to illustrate examples of pair checks and may include additional constraints, such as those discussed, above, in connection with FIG. 2.

(74) There are various techniques for providing the verification key $V_{\text{sub.K}}$ for use in connection with the validation of a proof π . FIG. 5 is an illustrative diagram 500 where a client 510 supplies the verification key $V_{\text{sub.K}}$ to be used in validation of a proof π . In an embodiment, a client creates a transaction output 502, wherein the transaction comprises or is otherwise associated with an output value 504 (e.g., payment of digital assets to a worker for contract execution) and a locking script 508 that performs a verification check comprising a bilinear mapping that satisfies a set of constraints. In an embodiment, the locking script verifies the integrity of the verification key $V_{\text{sub.K}}$ based at least in part on a digital signature of the client. In some cases, the locking script may specify that if the validation fails, the client is able to re-claim the digital assets as described above, in connection with the transaction. Such a transaction may be sent to a blockchain network to be mined by nodes of the blockchain network (e.g., workers). In some blockchain systems, the transaction encodes, in association with the locking script, a parameter indicating the size (e.g., in bytes) of the locking script, which may be referred to as the locking script size 506, such as is illustrated in connection with FIG. 5.

(75) As an example, a locking script may be described based on the following:

(76) TABLE-US-00002 OP_IF //verify the integrity of the $V_{\text{sub.K}}$ <PubKey Alice> OP_CHECKSIGVERIFY OP_0 OP_PICK OP_DUP < $V_{\text{sub.K}}^{\text{sup.6}}$ > OP_EQUALVERIFY OP_1 OP_SUB OP_PICK OP_DUP < $V_{\text{sub.K}}^{\text{sup.5}}$ > OP_EQUALVERIFY OP_([$V_{\text{sub.K}}$] - i) OP_PICK OP_DUP < $V_{\text{sub.K}}^{\text{sup.i}}$ > OP_EQUALVERIFY //verify the pairings // e($\pi_{\text{sub.2}}, V_{\text{sub.K}}^{\text{sup.2}}$) = e($\pi_{\text{sub.1}}, V_{\text{sub.K}}^{\text{sup.3}}$) //left part OP_DEPTH OP_3 OP_SUB OP_PICK OP_5 OP_PICK OP_PAIRING OP_TOALTSTACK //right part OP_DEPTH OP_2 OP_SUB OP_PICK OP_4 OP_PICK OP_PAIRING OP_TOALTSTACK //empty the stack 14*times {OP_DROP} OP_FROMALTSTACK OP_FROMALTSTACK OP_EQUALVERIFY [1] //for e($\pi_{\text{sub.i}}, V_{\text{sub.K}}^{\text{sup.j}}$) OP_DEPTH OP_(i+1) OP_SUB OP_PICK OP_([$V_{\text{sub.K}}$] - j + 1) OP_PICK OP_PAIRING OP_TOALTSTACK //for e($V_{\text{sub.K}}^{\text{sup.j}}, \pi_{\text{sub.i}}$) OP_([$V_{\text{sub.K}}$] - j) OP_PICK OP_DEPTH OP_(i+1) OP_SUB OP_PICK OP_PAIRING OP_TOALTSTACK <PubKey Bob> OP_CHECKSIG OP_ELSE <n days> OP_CHECKSEQUENCEVERIFY <PubKey Alice> OP_CHECKSIG OP_ENDIF

(77) It should be noted that the example provided above is descriptive of a locking script and not necessarily, verbatim, a locking script 508 itself. For example, some fields above that are described in brackets—such as “<PubKey Client>” which may refer to the client's public key—are not included, verbatim, in a locking script 508. Similarly, some fields in parentheses may be based on a mathematical computation—for example, “OP_(i+1)” as described above may not be included, verbatim, in a locking script, but rather refers to an opcode or command that is determined based at least in part on a mathematical computation—in this case, ‘i’ refers to the number of elements of proof π and, accordingly, in an example where proof $\pi = \{\pi_{\text{sub.1}}, \dots, \pi_{\text{sub.8}}\}$ “OP_(i+1)” may, in a locking script, be represented as “OP_7” in a stack-based scripting language. It should further be noted that text following the double slashes “//” above refer to comments which do not correspond to executable code. For example, “//verify the pairings” does not map to executable code and merely indicates, to a human, that the text following to comment is utilized to perform a verification of the pairings.

(78) It should be noted, again, that the transaction output 502 as described above is merely illustrative and that such variations may exist—the transaction may include any suitable payment amount in any suitable unit of measurement, and that various locking scripts may exist, such as those that utilize some or all functionality of locking scripts as described above. Generally speaking, a locking script 508 may be any suitable set of commands that performs a set of pairing check. In an embodiment, the locking script 508 includes an integrity check of the verification key $V_{\text{sub.K}}$ using at least an asymmetric public key associated with the client. In an embodiment, a locking script allows a client to reclaim digital assets if the validation fails. In an embodiment, the locking script 508 includes instructions to mark the transaction as invalid after a threshold duration (e.g., a value corresponding to “<n days>” as described above).

(79) The client 510, which may be a client such as those described elsewhere in this disclosure, may create a transaction input 512 that references the transaction 502 described above and transmits it to a counterparty (e.g., a worker that will purported compute a valid proof π). In an embodiment, the transaction input 512 may encode an identifier 514 associated with the transaction output 502, a transaction output index 516 associated with the transaction output 502 (e.g., in some cases, the index is zero-based), an unlocking script 520, a parameter indicating the size (e.g., in bytes) of the unlocking script, which may be referred to as the unlocking script size 518, a sequence number 522, and any suitable combination thereof. For example, in some embodiments, the unlocking script size is not explicitly encoded in the transaction input 512 and is otherwise derivable (e.g., through the detection of a specific terminating sequence that indicates the end of the unlocking script. In an embodiment, the unlocking script 518 comprises the verification key $V_{\text{sub.K}}$ 524 and a digital signature 526 associated with the client. In some cases, the unlocking script 518 includes

additional information, such as an indication of branching information that controls the execution of a verification process.

(80) As an example, an unlocking script may be described based on the following: <V.sub.K.sup.1><V.sub.K.sup.2><V.sub.K.sup.3><V.sub.K.sup.4><V.sub.K.sup.5><V.sub.K.sup.6><Sig A>1

(81) It should be noted that the example provided above is descriptive of an unlocking script and not necessarily, verbatim, an unlocking script itself. For example, some fields above that are described in brackets are not included, verbatim, in an unlocking script **518**. Returning to the example above, which is descriptive of an unlocking script, an unlocking script **518** may comprise an ordered or unordered sequence of elements of a verification key V.sub.K **524**, a digital signature **526** associated with the client, and branching information. In an embodiment, the digital signature **526** includes a hash type flag. In an embodiment, the hash flag is SIGHASH_NONE|SIGHASH_ANYONECANPAY in accordance with a Bitcoin-based system. An unlocking script **518** may comprise execution control information that is usable in conjunction with the locking script to validate a transaction. For example, in the example described above, the “1” indicates that the script should go into the first branch of a control statement in an unlocking script (e.g., an OP_IF branch instead of an OP_ELSE branch).

(82) In an embodiment, the worker **528** computes a proof π **530** and appends it with a digital signature **532** associated with the worker to the unlocking script **518** and broadcasts the transaction (e.g., to the client and/or one or more nodes of a blockchain network).

(83) As an example, an unlocking script having the proof and the worker's digital signature may be described based on the following: <Sig B>< π .sub.1>< π .sub.2> . . . < π .sub.8><V.sub.K.sup.1><V.sub.K.sup.2><V.sub.K.sup.3><V.sub.K.sup.4><V.sub.K.sup.5><V.sub.K.sup.6>V> <Sig A>1

(84) Accordingly, in an embodiment, an unlocking script in accordance with the description immediately above is able to satisfy the conditions placed on the output by the locking script and may allow the digital assets (e.g., as indicated by an output value) to be spent.

(85) There are various techniques for providing the validation key V.sub.K for use in connection with the validation of a proof π . FIG. **6** is an illustrative diagram **600** where a worker supplies the verification key V.sub.K to be used in validation of a proof π . The transaction output **602** may be in accordance with a Bitcoin-based system and may comprise an output value **604**, a locking script **606** (optionally, in some systems), and a locking script **608**. In an embodiment, the transaction is a Pay-to-Script-Hash (P2SH) transaction in accordance with a Bitcoin-based system. In an embodiment, the transaction output **612** is in accordance with a Bitcoin-based system and comprises a transaction ID **614**, an output index **616**, an unlocking script size **618**, an unlocking script **618**, and a sequence number **622** as described in connection with FIG. **5** above. It should be noted that while the structure of a transaction described in FIG. **6** may be in accordance with those described in FIG. **5**, FIG. **6** may include, in particular, a locking script **608** and unlocking script **618** in accordance with a P2SH transaction. The client **610** may generate the verification key V.sub.K and the redeem script **628** and provide them to the worker **624**.

(86) In an embodiment, a blockchain system supports various types of transactions. In an embodiment, a supported transaction (e.g., a standard transaction) is a script-hash-based transaction such as a Pay-to-Script-Hash (P2SH) transaction in accordance with a Bitcoin-based system. Generally speaking, a script-hash-based transaction refers to any transaction wherein verifying the validity of an unlocking script includes providing a script that matches a specified hash value. For example, in a Bitcoin-based P2SH transaction, an unlocking script **620** comprises a redeem script **628** and a locking script **608** comprises at least one condition that the hash of the redeem script supplied by the unlocking script matches a specified value. For example, in a Bitcoin-based system, a locking script may be described based on the following: OP_HASH160<20-byte hash of redeem script>OP_EQUAL

(87) In some cases, the worker **624** provides the verification key V.sub.K **626** to be used in the validation phase. The unlocking conditions that check the Validation Phase can, in an embodiment, be stored in a redeem script. In an embodiment, the locking script of the P2SH transaction comprises a hash of the redeem script and in some cases, the redeem script is kept secret (e.g., encrypted by the worker) and is only revealed upon an indication to cause the transfer of the output value.

(88) In an embodiment, a client **610** creates a P2SH unspent transaction output by identifying a redeem script and applies a hash to it (e.g., HASH160). To transfer the UTXO, the client creates an input script (which references the UTXO) that includes and/or otherwise references the redeem script. It should be noted that, in an embodiment, the redeem script is suitable to store arbitrary data, although a blockchain-based system may have restrictions on the type or amount of data that may be included in a redeem script. For example, in connection with Bitcoin-based systems there may be limitations on the size of data that can be published to the stack (e.g., a PUSHDATA operation is limited to 520 bytes of data). In an embodiment, a redeem script comprises compressed and uncompressed points P.sub.C and P.sub.U, respectively, as described in connection with FIG. **4**. Accordingly, in an embodiment, the maximum number of elliptic curve points that can be stored in a redeem script in accordance with a Bitcoin-based protocol is 15 compressed (e.g., 15 points*34 bytes/point=510 bytes) or 7 uncompressed points (7 points*66 bytes/point=462 bytes). In an embodiment, the unlocking script **618** comprises the proof **630**, the redeem script **628**, and commands (e.g., opcodes) that may, collectively, be used to satisfy a set of conditions encoded in the locking script **608**.

(89) FIG. **7** is an illustrative diagram of a process **700** for generating a redeem script in accordance with an embodiment. In an embodiment, the process **700** is implemented using hardware, software, or a combination thereof. A suitable system for performing the process includes a worker that supplies a P2SH unlocking script in connection with the discussions of FIG. **6**.

(90) In an embodiment, the system determines the verification key V.sub.K is of a sufficient cardinality. In an embodiment, the system determines **702** the size of a redeem script comprising each element of the verification key and a script to check a set of constraints, wherein the redeem script and a proof π are sufficient to unlock a corresponding locking script. The system may determine whether **704** the size of such a redeem script exceed a predetermined threshold, which may be based on a size limit imposed by the blockchain protocol—for example, a blockchain protocol may require the redeem script be less than or equal to 520-bytes in size. If redeem script is of a sufficient size, the system generates **706** a redeem script and unlocking script wherein the redeem script comprises the verification key and a script to check a set of constraints and the unlocking script comprises a proof π , wherein the redeem script and unlocking script together comprise a set of commands sufficient to validate the payment transaction.

(91) For example, in the case where the cardinality of the verification key V.sub.K is smaller than 16 it may be that:

(92) TABLE-US-00003 Redeem Script <V.sub.K.sup.1> <V.sub.K.sup.2> <V.sub.K.sup.3> <V.sub.K.sup.4> <V.sub.K.sup.5> <V.sub.K.sup.6>
OP_DEPTH OP_3 OP_SUB OP_PICK OP_5 OP_PICK OP_PAIRING OP_TOALTSTACK OP_DEPTH OP_2 OP_SUB OP_PICK OP_4
OP_PICK OP_PAIRING OP_TOALTSTACK 14*{OP_DROP} OP_FROMALTSTACK OP_FROMALTSTACK OP_EQUALVERIFY [1]
OP_DEPTH OP_5 OP_SUB OP_PICK OP_5 OP_PICK OP_PAIRING OP_TOALTSTACK OP_1 OP_PICK OP_DEPTH OP_3 OP_SUB
OP_PICK OP_PAIRING OP_TOALTSTACK 14*{OP_DROP} OP_FROMALTSTACK OP_FROMALTSTACK OP_EQUALVERIFY [2]
Unlocking Script OP_1 < π .sub.1> < π .sub.2> . . . < π .sub.8> <Redeem Script> Locking Script OP_HASH160 <20-byte-hash of [Redeem Script]>
OP_EQUAL

wherein the redeem script and unlocking script are sufficient to unlock the locking script.

(93) If the redeem script exceeds the predetermined threshold, the system generates **708** an unlocking script that stores one or more elements of the verification key V.sub.K in the part of the unlocking script that precedes the redeem script so that the total size of the redeem script is within the threshold and generates **710** a redeem script with the elements of the verification key and the constraints as described above. In an embodiment, the one or more elements verification key V.sub.K may be encoded elsewhere in the unlocking script. For example, in the case where the cardinality of V.sub.K is greater than 15 it may be that:

(94) TABLE-US-00004 Redeem Script OP_DUP OP_TOALTSTACK OP_HASH160 <20-byte-hash of V.sub.K.sup.6> OP_EQUALVERIFY
OP_DUP OP_TOALTSTACK OP_HASH160 <20-byte-hash of V.sub.K.sup.5> OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_HASH160
<20-byte-hash of V.sub.K.sup.4> OP_DUP OP_TOALTSTACK OP_HASH160 <20-byte-hash of V.sub.K.sup.3> OP_EQUALVERIFY OP_DUP
OP_TOALTSTACK OP_HASH160 <20-byte-hash of V.sub.K.sup.2> OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_HASH160 <20-byte-

hash OP_Vsub.K.sup.1> OP_EQUALVERIFY OP_FROMALTSTACK OP_FROMALTSTACK OP_DEPTH OP_3 OP_SUB OP_PICK OP_5 OP_PICK OP_PAIRING OP_TOALTSTACK OP_DEPTH OP_2 OP_SUB OP_PICK OP_4 OP_PICK OP_PAIRING OP_TOALTSTACK 14*{OP_DROP} OP_FROMALTSTACK OP_FROMALTSTACK OP_EQUALVERIFY [1] Unlocking Script OP_1 < π .sub.1> < π .sub.2> . . . < π .sub.8> <V.sub.K.sup.1> <V.sub.K.sup.2> <V.sub.K.sup.3> <V.sub.K.sup.4> <V.sub.K.sup.5> <V.sub.K.sup.6> <Redeem Script> Locking Script OP_HASH160 <20-byte-hash of [Redeem Script]> OP_EQUAL

wherein the redeem script and unlocking script are sufficient to unlock the locking script.

(95) Thus, by encoding the stages necessary in the Validation phase in a locking script, the validation of the transaction should become an equivalent process with the Validation phase of a zk-protocol. In an embodiment, an OP_VERIFYPROOF op code can be used as follows:

(96) TABLE-US-00005 Type Script Unlocking Script OP_1 < π .sub.1> < π .sub.2> . . . < π .sub.8> <V.sub.K.sup.1> <V.sub.K.sup.2> <V.sub.K.sup.3> <V.sub.K.sup.4> <V.sub.K.sup.5> <V.sub.K.sup.6> Locking Script // extract π .sub.i OP_DEPTH OP_(i + 1) OP_SUB OP_PICK // extract V.sub.K.sup.j OP_(|V.sub.K|-j + 1) OP_PICK // pairing call OP_PAIRING OP_TOALTSTACK // extract π .sub.k OP_DEPTH OP_(k + 1) OP_SUB OP_PICK // extract V.sub.K.sup.m OP_(|V.sub.K|-m + 1) OP_PICK // pairing call OP_PAIRING OP_TOALTSTACK //empty the stack (|V.sub.K| + | π |) * OP_DROP // verify the result of the operation OP_FROMALTSTACK OP_FROMALTSTACK OP_EQUALVERIFY

(97) The unlocking and locking scripts may be implemented in any suitable manner. The unlocking script is implemented in any suitable manner wherein the unlocking script encodes < π .sub.1> . . . < π .sub.8> and the verification key V.sub.K. Similarly, the locking script may be implemented in any suitable manner in which π .sub.i and V.sub.K.sup.j are extracted and a pairing call is made. In an embodiment, OP_PAIRING as described above is an op code that supports elliptic curves with efficient bilinear mappings such as a bn128 curve defined by the equation $y_{sup.2} = x_{sup.3} + 3$.

(98) FIG. 8 is an illustrative, simplified block diagram of a computing device 800 that can be used to practice at least one embodiment of the present disclosure. In various embodiments, the computing device 800 can be used to implement any of the systems illustrated and described above. For example, the computing device 800 can be configured for use as a data server, a web server, a portable computing device, a personal computer, or any electronic computing device. As shown in FIG. 8, the computing device 800 could include one or more processors 802 that, in embodiments, communicate with and are operatively coupled to a number of peripheral subsystems via a bus subsystem 804. In some embodiments, these peripheral subsystems include a storage subsystem 806 comprising a memory subsystem 808 and a file/disk storage subsystem 810, one or more user interface input devices 812, one or more user interface output devices 814, and a network interface subsystem 816. Such storage subsystem 806 could be used for temporary or long-term storage of information.

(99) In some embodiments, the bus subsystem 804 provides a mechanism for enabling the various components and subsystems of computing device 800 to communicate with each other as intended. Although the bus subsystem 804 is shown schematically as a single bus, alternative embodiments of the bus subsystem utilize multiple busses. In some embodiments, the network interface subsystem 816 provides an interface to other computing devices and networks. The network interface subsystem 816, in some embodiments, serves as an interface for receiving data from and transmitting data to other systems from the computing device 800. In some embodiments, the bus subsystem 804 is utilized for communicating data such as details, search terms, and so on.

(100) In some embodiments, the user interface input devices 812 includes one or more user input devices such as a keyboard; pointing devices such as an integrated mouse, trackball, touchpad, or graphics tablet; a scanner; a barcode scanner; a touch screen incorporated into the display; audio input devices such as voice recognition systems, microphones; and other types of input devices. In general, use of the term “input device” is intended to include all possible types of devices and mechanisms for inputting information to the computing device 800. In some embodiments, the one or more user interface output devices 814 include a display subsystem, a printer, or non-visual displays such as audio output devices, etc. In some embodiments, the display subsystem includes a cathode ray tube (CRT), a flat-panel device such as a liquid crystal display (LCD), light emitting diode (LED) display, or a projection or other display device. In general, use of the term “output device” is intended to include all possible types of devices and mechanisms for outputting information from the computing device 800. The one or more user interface output devices 814 can be used, for example, to present user interfaces to facilitate user interaction with applications performing processes described and variations therein, when such interaction may be appropriate.

(101) In some embodiments, the storage subsystem 806 provides a computer-readable storage medium for storing the basic programming and data constructs that provide the functionality of at least one embodiment of the present disclosure. The applications (programs, code modules, instructions), when executed by one or more processors in some embodiments, provide the functionality of one or more embodiments of the present disclosure and, in embodiments, are stored in the storage subsystem 806. These application modules or instructions can be executed by the one or more processors 802. In various embodiments, the storage subsystem 806 additionally provides a repository for storing data used in accordance with the present disclosure. In some embodiments, the storage subsystem 806 comprises a memory subsystem 808 and a file/disk storage subsystem 810.

(102) In embodiments, the memory subsystem 808 includes a number of memories, such as a main random access memory (RAM) 818 for storage of instructions and data during program execution and/or a read only memory (ROM) 820, in which fixed instructions can be stored. In some embodiments, the file/disk storage subsystem 810 provides a non-transitory persistent (non-volatile) storage for program and data files and can include a hard disk drive, a floppy disk drive along with associated removable media, a Compact Disk Read Only Memory (CD-ROM) drive, an optical drive, removable media cartridges, or other like storage media.

(103) In some embodiments, the computing device 800 includes at least one local clock 824. The local clock 824, in some embodiments, is a counter that represents the number of ticks that have transpired from a particular starting date and, in some embodiments, is located integrally within the computing device 800. In various embodiments, the local clock 824 is used to synchronize data transfers in the processors for the computing device 800 and the subsystems included therein at specific clock pulses and can be used to coordinate synchronous operations between the computing device 800 and other systems in a data centre. In another embodiment, the local clock is a programmable interval timer.

(104) The computing device 800 could be of any of a variety of types, including a portable computer device, tablet computer, a workstation, or any other device described below. Additionally, the computing device 800 can include another device that, in some embodiments, can be connected to the computing device 800 through one or more ports (e.g., USB, a headphone jack, Lightning connector, etc.). In embodiments, such a device includes a port that accepts a fibre-optic connector. Accordingly, in some embodiments, this device is that converts optical signals to electrical signals that are transmitted through the port connecting the device to the computing device 800 for processing. Due to the ever-changing nature of computers and networks, the description of the computing device 800 depicted in FIG. 8 is intended only as a specific example for purposes of illustrating the preferred embodiment of the device. Many other configurations having more or fewer components than the system depicted in FIG. 8 are possible.

(105) It should be noted that the above-mentioned embodiments illustrate rather than limit the invention, and that those skilled in the art will be capable of designing many alternative embodiments without departing from the scope of the invention as defined by the appended claims. In the claims, any reference signs placed in parentheses shall not be construed as limiting the claims. The word “comprising” and “comprises”, and the like, does not exclude the presence of elements or steps other than those listed in any claim or the specification as a whole. In the present specification, “comprises” means “includes or consists of” and “comprising” means “including or consisting of”. The singular reference of an element does not exclude the plural reference of such elements and vice-versa. The invention may be implemented by means of hardware comprising several distinct elements, and by means of a suitably programmed computer. In a device claim enumerating several means, several of these means may be embodied by one and the same item of hardware. The mere fact that certain measures are recited in mutually different dependent claims does not indicate that a combination of these measures cannot be used to advantage.

Claims

1. A computer-implemented method of controlling transfer of a digital asset via a blockchain, the method comprising: generating a transaction output of a transaction comprising an indication of a digital asset and a locking script that encodes a set of conditions for transferring control of the digital asset, satisfaction of the set of conditions to be determined based at least in part on a verification key and a proof; generating a transaction input of the transaction, the transaction input comprising: an identifier associated with the transaction output; and an unlocking script comprising the proof and a redeem script; and verifying the set of conditions are satisfied based at least in part on the locking script and the unlocking script; transferring control of the digital asset in response to verifying satisfaction of the set of conditions; determining a size of the redeem script; if the size of the redeem script does not exceed a predetermined threshold, generating the redeem script comprising the verification key and a script to check a set of constraints; and if the size of the redeem script exceeds the predetermined threshold, generating the unlocking script that stores one or more elements of the verification key in the part of the unlocking script preceding the redeem script so that the size of the redeem script is within the predetermined threshold, and generating the redeem script with the remaining elements of the verification key and the script to check a set of constraints.
 2. The computer-implemented method according to claim 1, wherein the verification key comprises a first plurality of elements of a finite field and the proof comprises a second plurality of elements of the finite field.
 3. The computer-implemented method according to claim 2, wherein elements of the finite field are points on an elliptic curve.
 4. The computer-implemented method according to claim 3, wherein the elements of the finite field are encoded in a compressed format.
 5. The computer-implemented method according to claim 1, wherein a client encodes the transaction input with the verification key and a first digital certificate associated with the client and a worker encodes the transaction input with the proof and a second digital certificate associated with the worker.
 6. The computer-implemented method according to claim 1, wherein the locking script comprises instructions that, contingent upon a condition of the set of conditions being unsatisfied by the unlocking script, reclaim the digital asset for provider of the digital asset.
 7. The computer-implemented method according to claim 6, wherein: wherein the verification key and the redeem script are to satisfy the set of conditions; and the locking script encodes a condition of the set of conditions that a hash of the redeem script matches a predetermined value.
 8. The computer-implemented method according to claim 7, wherein the redeem script is less than or equal to 520 bytes in size.
 9. The computer-implemented method according to claim 7, wherein the unlocking script comprises one or more elements of the verification key.
 10. The computer-implemented method according to claim 7, wherein the transaction is a P2SH transaction in accordance with a Bitcoin based protocol.
 11. The computer-implemented method according to claim 1, wherein the transaction is a standard transaction in accordance with a blockchain-based protocol.
 12. The computer-implemented method according to claim 1, wherein the locking script and unlocking script are encoded in a stack-based scripting language.
 13. The computer-implemented method according to claim 1, wherein the set of conditions comprise a bilinear constraint.
 14. A system, comprising: a processor; and memory including executable instructions that, as a result of being executed by the processor, cause the system to perform the computer-implemented method according to claim 1.
 15. A non-transitory computer-readable storage medium having stored thereon executable instructions that, as a result of being executed by a processor of a computer system, cause the computer system to at least perform the computer-implemented method according to claim 1.
-